REAL-TIME PHISHING URL DETECTION USING ML AND DL

A Project Report

Submitted in partial fulfillment of the

Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Shaikh Shamsuddin Mohiuddin

3010205

Under the esteemed guidance of

Miss. Safa Momin

DEPARTMENT OF INFORMATION TECHNOLOGY

Padmashri Annasaheb Jadhav Bhartiya Samaj Unnati Mandal's

BNN COLLEGE (ARTS,SCIENCE AND COMMERCE)

(Affiliated to University of Mumbai)

BHIWANDI- 421 302

MAHARASHTRA 2025-2026

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PRN NO: **2023016400242625** Seat No: **3010205**

1. Name of the Student: **Shaikh Shamsuddin Mohiuddin**
2. Title of the Project: **REAL-TIME PHISHING URL DETECTION USING ML AND DL**
3. Name of the Guide: **Miss. Safa Momin**
4. Teaching experience of the Guide: **5**
5. Is this your first submission? **Yes**

Signature of the Student

Date:

Signature of the Guide

Date:

Signature of the Coordinator

Date:

Padmashri Annasaheb Jadhav Bhartiya Samaj Unnati Mandal's

BNN COLLEGE (ARTS,SCIENCE AND COMMERCE)

(*Affiliated to University of Mumbai)*

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled, "**REAL-TIME PHISHING URL DETECTION USING ML AND DL"** is bonafied work of **SHAIKH SHAMSUDDIN MOHIUDDIN** bearing **Seat No: 3010205** submitted in partial fulfillment of the requirement for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Internal Guide **Coordinator**

External Examiner

Date: **College Seal**

ABSTRACT

Phishing is one of the most common cyber threats, where malicious websites trick users into revealing sensitive information such as passwords, banking details, and personal data. With the rapid growth of online services, detecting phishing websites in real-time has become a critical challenge for ensuring cybersecurity.

The proposed project focuses on developing a ***Real-Time Phishing URL Detection System using Machine Learning and Deep Learning*** techniques. Various URL-based features, content-based features, and lexical analysis are performed to classify whether a given URL is legitimate or phishing. Advanced algorithms such as Random Forest, Logistic Regression, and Deep Neural Networks are implemented to achieve higher accuracy.

The system will provide fast, reliable, and automated phishing detection that can be integrated into browsers or security tools. This project aims to protect users from cyber frauds, identity theft, and financial losses by enabling secure browsing in real time.

Keyword:

1. **Phishing** – fraudulent URLs/websites tricking users.
2. **URL Detection** – analysing and classifying URLs.
3. **Real-Time Detection** – immediate analysis of submitted URLs.
4. **Machine Learning (ML)** – algorithms for classification and prediction.
5. **Deep Learning (DL)** – neural networks for advanced detection.
6. **Cybersecurity** – protecting users from online threats.
7. **Feature Extraction** – identifying URL characteristics (length, domain, symbols, etc.).
8. **Classification** – categorizing URL as (*Safe* or *Phishing)*.
9. **Prediction Model –** ML/DL model generating the output.
10. **Report Generation** – providing logs and results to user/admin.

ROLE AND RESPONSIBILITY FORM

Role

My role was to design and develop the complete system titled *"Real-Time Phishing URL Detection using ML and DL"*. I was responsible for conducting a thorough **requirement analysis**, **designing the system architecture** and **implementing core modules** such as user authentication, URL analysis, and report generation. Additionally, I ensured that the system achieved its primary objectives of being **secure, reliable, and capable of performing real-time phishing URL detection**.

Responsibility

Our main Motive for end-to-end project development including analysis, design, coding (Machine Learning/Deep Learning model development), testing, documentation, and final presentation. I implemented the machine learning and deep learning models for URL feature extraction and prediction, ensuring accurate and real-time phishing detection. Additionally, I developed the front-end and back-end modules, integrated the prediction results with user-friendly reporting features, and maintained secure authentication mechanisms.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to everyone who supported me throughout this project. First, I thank my mentors and advisors **Miss. Safa Momin** for their guidance and encouragement during the planning, development, and completion **REAL-TIME PHISHING URL DETECTION USING ML AND DL** of Their valuable feedback helped me improve the project at each stage.

I have been greatly benefited from their regular critical reviews and inspirations throughout my work. I am grate full to other for their guidance, encouragement, understanding, and insight full support in the development process

Now, I would like to express my sincere thanks to our **I/C Principal Prof. Dr. Shashikant R. Mhalunkar Sir** and course Co-ordinate **Mr. Pramod Lala Shewale Sir** for having facilitated us with the essential infrastructure and resources without which the project would not have seen the sunrise. I am also thankful to entire staff of Information Technology for their constant encouragement, suggestions, and moral support throughout the duration of my project

Last but not the least, I would like to mention here that I am greatly indebted to each and everybody my friends and anyone who has been associated with my project at any stage but their name could not find a place in this acknowledgement.

DECLARATION

I hereby declare that the project entitled, "**REAL-TIME PHISHING URL DETECTION USING ML AND DL"** done at **B.N.N College**. The work has been carried out with sincerity, and all the data and information provided in this report are true to the best of my knowledge. This report is a result of independent research and development, and any sources of information or data have been duly acknowledged.

The project is done in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of our curriculum.

SHAIKH SHAMSUDDIN MOHIUDDIN

TABLE OF CONTENTS

Sr No.	Title	Page No.
Chapter 1	**Introduction**	1
	1.1 Background	2
	1.2 Objectives	3
	1.3 Purpose, Scope, and Applicability	4-5
	1.3.1 Purpose	4
	1.3.2 Scope	4
	1.3.3 Applicability	5
	1.4 Achievements	5
	1.5Organisation of Report	6
Chapter 2	**Survey of Technologies**	7-9
Chapter 3	**Requirements and Analysis**	10
	3.1 Problem Definition	11
	3.2 Requirements Specification	12
	3.3 Planning and Scheduling	13-14
	3.4 Software and Hardware Requirements	15
	3.5 Preliminary Product Description	16-17
	3.6 Conceptual Models	18-19
Chapter 4	**System Design**	20
	4.1 Basic Modules	21-22
	4.2 Data Design	23
	4.2.1 Schema Design	23-24
	4.2.1 Data Integrity and Constraints	25-26
	4.3 Procedural Design	27-28
	4.3.1 Logic Diagrams	29-32
	4.3.2 Data Structures	33-35
	4.3.3 Algorithms Design	36-37
	4.4 User interface Design	38-39
	4.5 Security Issues	40-41
	4.6 Test Cases Design	42-43
Chapter 5	**Implementation and Testing**	44
	5.1 Implementation Approaches	45-46
	5.2 Coding detail and Code Efficiency	47
	5.2.1 Coding detail	47-62

	5.2.2 Code Efficiency	62-64
	5.3 Testing Approach	65
	5.3.1 Unit Testing	66
	5.3.2 Integrated Testing	67
	5.3.3 Beta Testing	68
	5.4 Modifications and Improvements	69
	5.5 Test Case	70
Chapter 6	**Results and Discussion**	71-79
	6.1 Test Reports	80
	6.2 User Documentation	81
Chapter 7	**Conclusions**	82
	7.1 Conclusion	83
	7.1.1 Significance of the System	84
	7.2 Limitations of the System	85
	7.3 Future Scope of the Project	86
	References	87
	Glossary	88-89
	Appendix A	90-91
	Appendix B	92-93

TABLE OF FIGURES

SR NO	TITLE	PAGENO
1	Fig 3.3.1 Gantt Chart	14
2	Fig 4.3.1.1 Data Flow Diagram	30
3	Fig 4.3.1.2 Entity Relationship Diagram	31
4	Fig 4.3.1.3 State Chart Diagram	32
5	Fig 4.3.2.1 Sequence Diagram	33
6	Fig 4.3.2.2 Use Case Diagram	34
7	Fig 4.3.2.3 Activity Diagram	35

CHAPTER 1:

INTRODUCTION

1.1 Background

In the Rapid growth of the internet , online services such as e-commerce, online banking, social media platform networking area and digital communication have certain essential our daily life used. However, this widespread digitalization increased the risk of the cybercrimes. Among various threatening, phishing attack emerged as one of the most common dangerous security challenges of our daily life **Phishing URL** is fraudulent multiple attackers trick creating fake websites or fake URL appears to be legitimate in order sensitive information such as username, passwords, or online banking details

Traditional phishing URL detection system rely heavily blacklist and rule-based approaches which can identify only previously known malicious URLS. These methods often fails when fail when attacker generate new phishing link or modify existing ones. Its overcome these limitations, researchers have shifted towards **Machine Learning (ML)** and **Deep Learning (DL)** based models, which can learn patterns from large datasets and detecting the phishing URLs more effectively

Behavioural analysis adds certain another dimension by studying **how users interact with URLs and websites**, identifying suspicious behaviour such as redirection, pop-ups, abnormal login requests. Integrating behavioural patterns with Machine Learning/Deep Learning models enhances the accuracy of phishing detection in real time.

Some focused on extracting **URL-based features** (length of URL, presence of "@" symbol, number of dots, etc.) While applied **content-based analysis** of webpages. Recent works emphasize the use of **hybrid models** that combine multiple feature sets and deep learning algorithms such as (CNNs, RNNs, or LSTMs) for better detection accuracy. Despite these advancements, phishing remains a persistent challenge due to the **ever-evolving tactics of attackers.**

1.2 Objectives

The primary objective in this project to design and implemented **A real-time phishing URL detection system** that leverages power of Machine Learning (ML) and Deep Learning (DL) algorithms for improved accuracy and efficiency. The project aims to extract lexical, structural, and domain-based features from URLs, analyser them through for Advanced Machine Learning and Deep Learning models, and classify them as either legitimate or phishing in nature.

Another objective is the incorporate **behavioural analysis**, which studies suspicious user interactions such as redirections, abnormal login requests, and pop-up triggers, thereby strengthening the detection mechanism beyond traditional URL-based detection. By combining feature-driven learning and behavioural insights the project seeks to minimize false positives and maximize the early detection of malicious true detection and highlight its attempts.

The project aims to provide an **interactive and user-friendly platform** capability of alert users in real time, Ensuring safer browsing experiences and contributing towards the more accurate secure digital ecosystem.

1.3 Purpose, Scope and Applicability

1.3.1 Purpose:

The main purpose of this project to developed a real-time phishing URL detection system that protects users from falling prey to fraudulent websites. Phishing has become one of the most common cyber threats, targeting sensitive information such as Login Credentials, Banking Details, and personal data. In Traditionally methods such as Blacklisted URLs and rule-based approaches are often ineffective against newly generated phishing URLs.

To Overcomes these limitations, The proposed system leverages Machine Learning (ML) and Deep Learning (DL) techniques for accurate classification of URLs. The Additionally integrating behavioural analysis, and design the project aims to identify suspicious activities like abnormal redirections, fake login requests, and unauthorized data collection. The purpose is not only to detect phishing attempts but also to provide real-time alerts, enabling users to browse the internet securely and confidently.

1.3.2 Scope:

The scope of this project includes the **end-to-end design and implementation** of a phishing URL detection system. It involves dataset collection, preprocessing of URLs, extraction of lexical and structural features and training Machine Learning/Deep Learning models. The project also extends its scope by incorporating behavioural analysis. which allows the system to detect advanced phishing techniques beyond static URL features.

The project scope covers diverse domains where phishing is the major concerned including **online net banking, e-commerce, online learning platforms, and social networking sites**.

1.3.3 Applicability:

The proposed system has wide range of practical applications in both individual and organizational contexts. For individuals, it can be deployed as a browser extension or mobile application to provide instant alerts. When the phishing URL is detected for organization and the system can be integrated into corporate firewalls, email filtering systems, and enterprise-level security tools to protect employees and customers from phishing scams

Financial institutions, government portals, healthcare services, and educational platforms can benefit from its adoption by ensuring safe digital transactions and data protection. The system can serve as **Research and Educational** Tool for cybersecurity training, enabling students and professionals to understand real-world phishing threats and countermeasures. Its adaptability across multiple platforms makes is highly applicable for solution today's evolving cyber landscape.

1.4 Achievements:

The student gained extensive knowledge in this fields of cybersecurity, phishing attack detection, artificial intelligence techniques. The project provided hands-on experience in data preprocessing, feature engineering and the application of Machine Learning (ML) and Deep Learning (DL) algorithms for real-world problems.

The project successfully demonstrated that combining **URL feature analysis with behavioural analysis** leads to more accurate phishing detection. Unlike traditional blacklist-based approaches, the proposed system can **detect newly generated phishing URLs in real time**, thereby addressing one of the major challenges in the cybersecurity domain.

This work enhanced skills in **Python programming, model evaluation, visualization, and performance optimization**, while also deepening awareness of cyber threats and preventive measures.

1.5 Organization of Report

The project report is organized in a systematic manner to provide clarity and logical flow to the reader. It begins with an introduction, which outlines the background, objectives, purpose, and scope of the work. This is followed by a review of related research, where existing methods and approaches in phishing detection are discussed to highlight the research gap.

The subsequent sections describe the **system requirements, analysis, and design methodology**, explaining how the proposed solution was planned and structured. The report represent the **implementation details and methodology**, covering the algorithms, tools, and techniques used to build the system. After implementation, the **results and evaluation** are discussed, showcasing the performance of the developed system through experimental findings.

Finally, the report concludes with a **summary of contributions, achievements and the suggestions for future enhancements**, ensuring the work is contextualized within the broader field of cybersecurity.

CHAPTER 2:

SURVEY OF TECHNOLOGIES

SURVEY OF TECHNOLOGIES

The implementation of the project depends on the effective use of technologies, tools, and platforms. In this project, The combination of **programming languages, machine learning frameworks, deep learning libraries and supporting software tools** has been adopted. A brief survey of the major technologies used is presented below.

1. **Programming Language:** Python

   Python is chosen as the primary programming language due to its simplicity, readability, and rich ecosystem of libraries for **machine learning, data analysis, and cybersecurity research**. Its flexibility allows seamless integration of ML/DL models with web applications.

2. **Machine Learning Libraries**

   - **Scikit-learn:** Provides efficient tools for classification, regression, and data preprocessing. It is widely used for feature extraction and traditional ML algorithms like Decision Trees, Random Forest, and SVM.
   - **Pandas & NumPy:** Used for dataset manipulation, feature engineering, and numerical operations.

3. **Deep Learning Frameworks**

   - **TensorFlow & Keras:** Used for building and training deep learning models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), which enhance phishing URL detection accuracy.

4. **Web Development Tools**

   - **Flask/Django:** Lightweight frameworks to integrate the ML/DL models into a real-time web-based interface.
   - **HTML, CSS:** Used to build a simple and interactive front-end for user interaction

5. **Database:** MySQL

   MySQL is used to store datasets, logs of detected phishing URLs, and system output for evaluation purposes.

6. Supporting Tools

- **Jupyter Notebook / Google Colab:** For prototyping and testing ML/DL models.
- **Matplotlib & Seaborn:** For visualizing datasets, model accuracy, confusion matrices, and comparison graphs.

7. Behavioural Analysis Tools

For monitoring suspicious URL behaviours such as abnormal redirections, pop-ups, or fake login requests, custom Python scripts and pattern recognition techniques are used. These help in capturing real-time user interaction data for analysis.

CHAPTER 3:

REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

Phishing attacks are the one of the most common and dangerous cyber threats in the digital world. Attackers often create fake websites or URLs that closely resemble legitimate ones in order to deceive users into sharing sensitive information such as passwords, banking credentials, and personal data. Traditional security systems, such as blacklists or simple rule-based filters. often fail to detect newly generated phishing URLs because attackers frequently change domain names, hosting platforms, and obfuscation methods. This creates a serious need for a real-time, intelligent and adaptive system that can detect phishing URLs accurately and quickly before any damage occurs.

Sub-Problems

1. **Identification of URL Features:**
   Extracting meaningful features from URLs such as domain length, presence of special characters and use of sub-domains and HTTPS usage to distinguish between phishing and legitimate websites.
2. **Behavioural Analysis of URLs:**
   Studying how URLs behave in terms of redirections, server responses, and unusual traffic activity and which often indicates phishing attempts.
3. **Model Selection and Training:**
   Choosing suitable Machine Learning/Deep Learning algorithms (e.g. Random Forest, LSTM, CNN) and training them with large datasets to ensure accurate classification of URLs.
4. **Real-Time Detection:**
   Designing a system that not only predicts phishing URLs accurately but also provides instant detection with minimal delay.
5. **System Integration:**
   Integrating the detection model with a user-friendly interface and backend system, making it accessible for real-world use.

3.2 Requirement Specification

The purpose in this phase to clearly define the requirements of the **Real-Time Phishing URL Detection with Behavioural Analysis** system but independently methods or technologies used to implemented them. This ensures that the expectations from the system are properly understood before development.

System Requirement: Functional and Non-Functional

1. **Functional Requirements:**

- The system should allow users to **input URLs** for verification.
- The system should **extract relevant features** from the given URL and perform **behavioural analysis**.
- The system must classify URLs as **"Phishing" or "Legitimate"** in real time.
- The system should generate **detailed reports** of detected phishing URLs for analysis.
- The system should provide an **alert mechanism** to warn users about suspicious URLs.

2. **Non-Functional Requirements:**

- The system must ensure **real-time performance** with minimal detection delay.
- The system should be **accurate** with the reduced false positives/negatives.
- It should be **user-friendly** and easy to interact with through a simple frontend
- The system must be **scalable** to handle larger volumes of URLs.
- It should be **secure**, ensuring no sensitive user data is exposed during analysis.

3.3 Planning and Scheduling

Planning and scheduling are crucial aspects of software development as they ensure that the project is executed systematically and within the given timeframe. For the project ***"Real-Time Detection of Phishing URLs with Behavioural Analysis using ML and DL"*,** the planning process involved identifying all the necessary tasks and arranging them in logical order. Constraints such as dependency between tasks (e.g., model training cannot begin before dataset preparation) were also considered.

Scheduling ensures that the necessary resources such as hardware, software tools, and time are allocated efficiently. The schedule is represented using tools like Gantt charts which provide a clear overview of timelines and dependencies.

- **Phase 1: Research and Requirement Gathering** (1 month): Conduct literature survey on phishing detection methods. Identify gaps in existing systems and Define system requirements (functional & non-functional).
- **Phase 2: Planning and System Analysis** (1 month): Break down the project into smaller tasks. Perform system feasibility study. Analyse constraints, risks, and resources required. Finalize project scheduling (Gantt chart)
- **Phase 3: Frontend and Backed Development** (2 months): Develop the **Frontend** using HTML, CSS for user interface. Implement the **Backend** using Flask/Python for model integration. Establish secure communication between frontend and backend. Connect to the **database** for MySQL storage and results logging.
- **Phase 4: Testing and Feedback** (1 month): Conduct **unit testing, integration testing, and system testing**. Collect feedback from test users/peers.Fix bugs and improve performance. Ensure UI/UX improvements based on feedback.
- **Phase 5: Launch and Deployment** (1 month): Final deployment on server (local + cloud). Documentation of user manual & project report Demonstration & submission.

GANTT CHART:

ID	Task	Aug’17	Sep’29	Oct’29	Nov’29	Dec’29	Jan’29	Feb’27
1	Requirements Gathering							
2	Planning							
3	System Design							
4	Implementation							
5	Testing							
6	Deployment							

Fig. 3.3.1. Gantt Chart

3.4 Software and Hardware Requirement

Hardware Requirement

Processor Intel i5, 16 GB RAM, Hard Disk 500GB, Display 14 or higher 1366x786, stable broadband for dataset collection, training and testing

Software Requirement

Operating System: windows 10/11

Programming Language: python 3.8 or above

IDE/Editor: Jupyter Notebook/PyCharm /VS code

Frameworks & Libraries:

Scikit-learn,Tensor Flow/PyTorch, Pandas, NumPy, Matplotlib/ Seaboarn/ Plotly ,etc.

Database: MySQL

Browser: Google Chrome/Firefox(for dataset & testing)

3.5 Preliminary Product Description

The proposed system aims to develop a **real-time phishing URL detection application** that integrates **machine learning (ML) and deep learning (DL)** models with behavioural analysis techniques to accurately identify and prevent phishing attempts.

The primary **requirements** of the system are:

- Collect and preprocess URL datasets, including phishing and legitimate URLs.
- Extract key features such as domain age, IP-based patterns, redirection counts, and URL length.
- Implement Machine Learning/Deep Learning models (e.g., Random Forest, CNN, RNN) for classification.
- Perform **behavioural analysis** by examining user interaction patterns, URL redirects, and suspicious request behaviours.
- Provide an **interactive frontend** for users to input/test URLs.
- Ensure **real-time detection** with instant feedback.
- Generate reports and logs for further analysis by administrators.

The **functions and operations** of the system include:

- **Input Function** – Users or system admins can submit URLs for verification.
- **Detection Engine** – Extracts features and applies ML/DL classifiers to identify phishing tendencies.
- **Behavioural Analyzer** – Monitors user navigation behaviour, unusual redirects, and suspicious activities.
- **Output Function** – Provides prediction results (Phishing / Legitimate) along with probability scores.
- **Report Generation** – Summarizes detection history, statistics, and trends for future reference.

3.6: Conceptual Model

The project is represented using conceptual models. These models illustrate how data flows within the system, how different components interact, and what operations are allowed at each stage. They provide both **functional** and **structural** views of the application.

1.Data Flow Diagram (DFD)

- **Level 0 (Context Diagram):**
  - User enters URL → System → Detection Result (Phishing/Legit).
- **Level 1:**
  - Input Module → Feature Extraction → ML/DL Model → Behavioural Analysis → Output Module.

2. Entity-Relationship (ER) Diagram

- **Entities:**
  - User (inputs URL)
  - Admin (monitors logs & reports)
  - URL Database (stores phishing & legitimate URLs)
  - Detection Engine (ML/DL models)
  - Behavioural Logs (stores redirect & click behaviour)
- **Relationships:**
  - User → submits → URL
  - Detection Engine → classifies → URL
  - Admin → reviews → Reports

3. Object-Oriented Diagram (UML Class Diagram)

- **Classes:**
  - User (attributes: user ID, name, role; methods: submit URL,view Report)
  - URL (attributes: urlID, domain, features; methods: extractFeatures)
  - DetectionEngine (attributes: modelType; methods: classifyURL)
  - BehaviorAnalyzer (methods: analyzeRedirects, detectSuspiciousBehavior)
  - Report (attributes: reportID, timestamp; methods: generateReport)

4. System Flowchart

- Start → Input URL → Preprocessing → Feature Extraction → ML/DL Model → Behavioural Analysis → Result Generation → Display Result (Phishing/Legit) → End

CHAPTER 4:

SYSTEM DESIGN

4.1 Basic Modules

1. User Authentication Module

- **Functionality:**
  - User registration and login.
  - Authentication using username and password.
  - Session management (login/logout).
- **Purpose:** Provides a secure entry point for genuine users and prevents unauthorized access.

2. Input & Preprocessing Module

- **Functionality:**
  - Capture user input (text, comments, posts).
  - Clean the text (removal of stopword punctuation handling, tokenization).
  - Prepare the input for the machine learning model.
- **Purpose:** Converts raw input into a proper format to ensure accurate analysis.

3. Analysis & Detection Module

- **Functionality:**
  - Apply Machine Learning or Deep Learning models.
  - Detect toxicity, emotions, phishing, or fake content (depending on the project).
  - Generate output such as safe, harmful, suspicious, etc.
- **Purpose:** Core logic for intelligent decision-making.

4. Database Management Module

- **Functionality:**
  - Store user details (credentials, profiles).
  - Save comments/posts along with their analysis results.
  - Maintain reports, history, and alerts.
- **Purpose:** Organizes data systematically for storage and future retrieval or enhancement.

5. Report Generation Module

- **Functionality:**
  - Convert results into a readable format.
  - Generate graphs, charts, and PDFs.
  - Allow viewing history and downloading reports.
- **Purpose:** Provides users and admins with clear, visual, and useful outputs.

6. Admin Dashboard Module

- **Functionality:**
  - Monitor system activity (active users, reports, logs).
  - Manage users (block/unblock, assign roles).
  - Check system alerts and suspicious activity.
- **Purpose:** Offers centralized control for admins to monitor and manage the application.

4.2 Data Design

The Data design defines how information is structured, stored, and accessed within the platform. The data design includes the definition of entities and their relationships.

4.2.1 Schema Design:

Schema design defines the structure of the database and how data will be stored, related, and accessed. A well-defined schema ensures efficient data storage, retrieval, and management for both real-time detection and historical analysis.

1.Users

Attributes: user_id (PK), username, email, password_hash, role, created_at

Description: Stores user registration and login details.

2.URLs

Attributes: url_id(PK),url_text,domain,submission_time,label

Description: Stores URLs along with their label (Phishing/Legitimate).

3.URL_Features

Attributes: feature_id (PK), url_id (FK), length, num_dots, has_at_symbol, has_ip_address, ssl_valid, num_special_chars.

Description: Stores extracted features for each URL.

4.Predictions

Attributes: prediction_id (PK), url_id (FK), predicted_label, model_used, prediction_time

Description: Stores results of ML/DL model predictions.

5.Reports

Attributes: report_id (PK), user_id (FK), url_id (FK), report_generated_time, report_file_path

Description: Stores generated reports for user access.

Explanation:

- Users table ensures secure authentication and role management.
- URLs table keeps all submitted URLs and labels for training/testing.
- URL_Features stores preprocessed and extracted features for model input.
- Predictions tracks real-time detection outcomes with model reference.
- Reports allows historical tracking and downloadable report management.

4.2.2 Data Integrity and Constraints:

To maintain accuracy, reliability, and consistency, the following data integrity rules and constraints are applied:

1. **Primary Key Constraints:**
   - Ensures each record in a table is unique.
   - Example: user_id in Users, url_id in URLs.

2. **Foreign Key Constraints:**
   - Maintains referential integrity between tables.
   - Example: url_id in URL_Features references URLs.url_id.

3. **Not Null Constraints:**
   - Critical fields cannot be empty.
   - Example: username, email, password_hash, url_text, label.

4. **Unique Constraints:**
   - Ensures no duplicate entries where required.
   - Example: email in Users, url_text in URLs.

5. **Data Type Constraints:**
   - Ensures appropriate data formats.
   - Example: submission_time is of type DATETIME, ssl_valid is BOOLEAN.

6. **Check Constraints:**
    - Validates data according to project-specific rules.
    - Example: label can only be 'Phishing' or 'Legitimate'.

7. **Default Values:**
    - Sets default entries for some fields.
    - Example: role in Users defaults to 'user'.

8. **Validation at Application Level:**
    - Additional checks during URL submission:
        - Valid URL format.
        - Avoid duplicate submissions.
        - Prevent SQL injection through input sanitization.

4.3 Procedural Design

1.User Authentication Procedure

- **Steps:**
    1. User enters login credentials.
    2. System checks credentials against the Users table.
    3. If valid, create a session; else, display error.
- **Purpose:** Ensure only authorized users can access the system.

2.URL Submission Procedure

- **Steps:**
    1. User enters a URL for verification.
    2. System validates URL format.
    3. Check for duplicate submission in URLs table.
    4. Store URL in the database.
- **Purpose:** Capture user input in a structured manner for further analysis.

3.Data Preprocessing Procedure

- **Steps:**
    1. Extract features from submitted URLs (length, number of dots, special characters, presence of IP, etc.).
    2. Normalize and encode features for model input.
    3. Store processed features in URL_Features table.
- **Purpose:** Convert raw URLs into machine-readable features for ML/DL models.

4.ML/DL Analysis Procedure

- **Steps:**
    1. Fetch preprocessed features for a URL.
    2. Apply trained ML/DL model (e.g., Random Forest, XGBoost, LSTM, or CNN).
    3. Generate classification output: Phishing or Legitimate.
    4. Store results in Predictions table.
- **Purpose:** Make intelligent decisions regarding URL safety.

5.Report Generation Procedure

- **Steps:**
    1. Fetch historical predictions or specific URL analysis.
    2. Generate visualizations (charts, graphs) of phishing statistics.
    3. Compile results into downloadable PDF reports.
    4. Store report path in Reports table.
- **Purpose:** Provide clear and actionable insights to users and admins.

6.Admin Monitoring Procedure

- **Steps:**
    1. Track active users, URL submissions, and prediction history.
    2. Detect suspicious activity patterns.
    3. Manage users (block/unblock, assign roles).
- **Purpose:** Maintain system integrity and monitor for misuse or anomalies.

4.3.1 Logic Diagram

Logic diagrams are graphical representations that define the systematic flow of a procedure. They help in understanding, designing, and analysing the flow of control, data, and processes in a system. Such diagrams provide a clear visualization of how different modules interact, making it easier for programmers to implement and debug the system

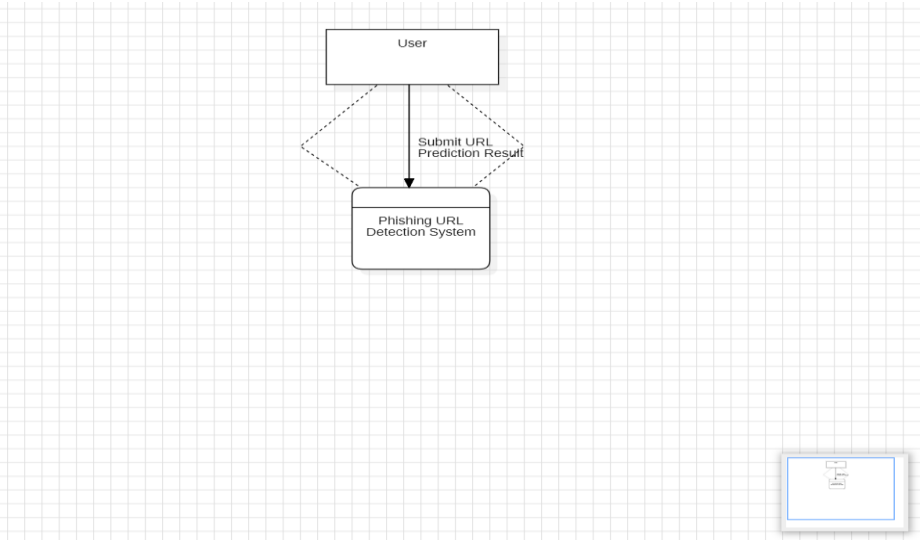
1. DATA FLOW DIAGRAM:

A Data Flow Diagram (DFD) is a graphical representation of how data flows through a system, showing the processes, data stores, and interactions between external entities and the system.

In the Real-Time Phishing URL Detection System, the DFD illustrates how user inputs (such as login credentials or suspicious URLs) are processed by different modules of the system. At Level-0, the DFD provides a high-level overview of the interaction between the user and the system, highlighting URL submission and result output.

At Level-1, the system is broken down into three major processes: User Authentication, URL Analysis, and Report Generation, along with connections to the User Database and Results Log. These diagrams help in understanding the logical flow of data, ensuring that each step from data input to analysis and final output is clearly defined and easily understandable.

DFD LEVEL-0:



DFD Level-1:

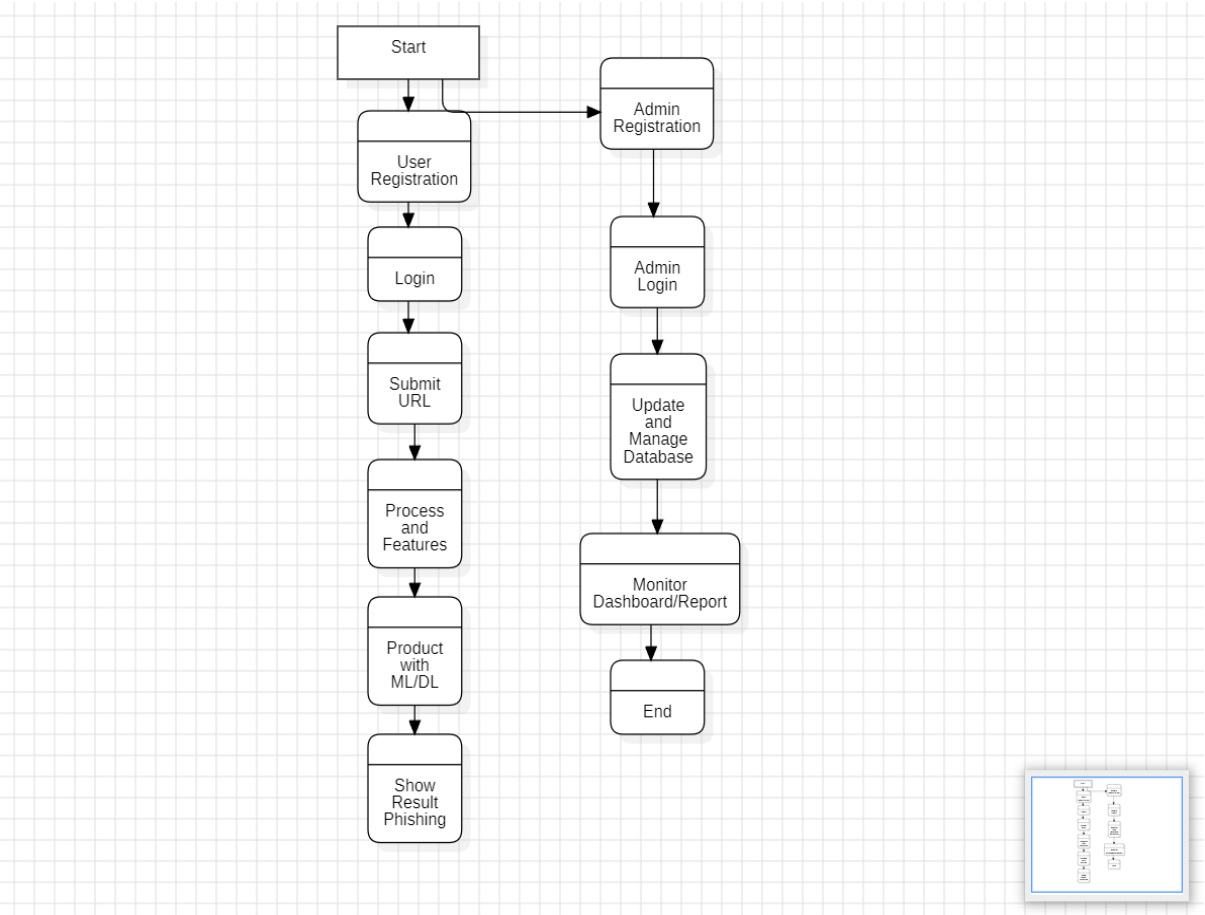


Fig.4.3.1.1. Data flow Diagram

2.ENTITY RELATIONSHIP DIAGRAM

An Entity Relationship Diagram (ERD) is a visual representation of the relationships between entities (or data objects) in a data base system. It helps to model the structure of data and its interactions, making it easier to design databases and understand how different elements relate to one another.

1. Entities: These represent objects or concepts.
2. Attributes: These describe the properties of an entity.
3. Relationships: These description the entities are related to each other.

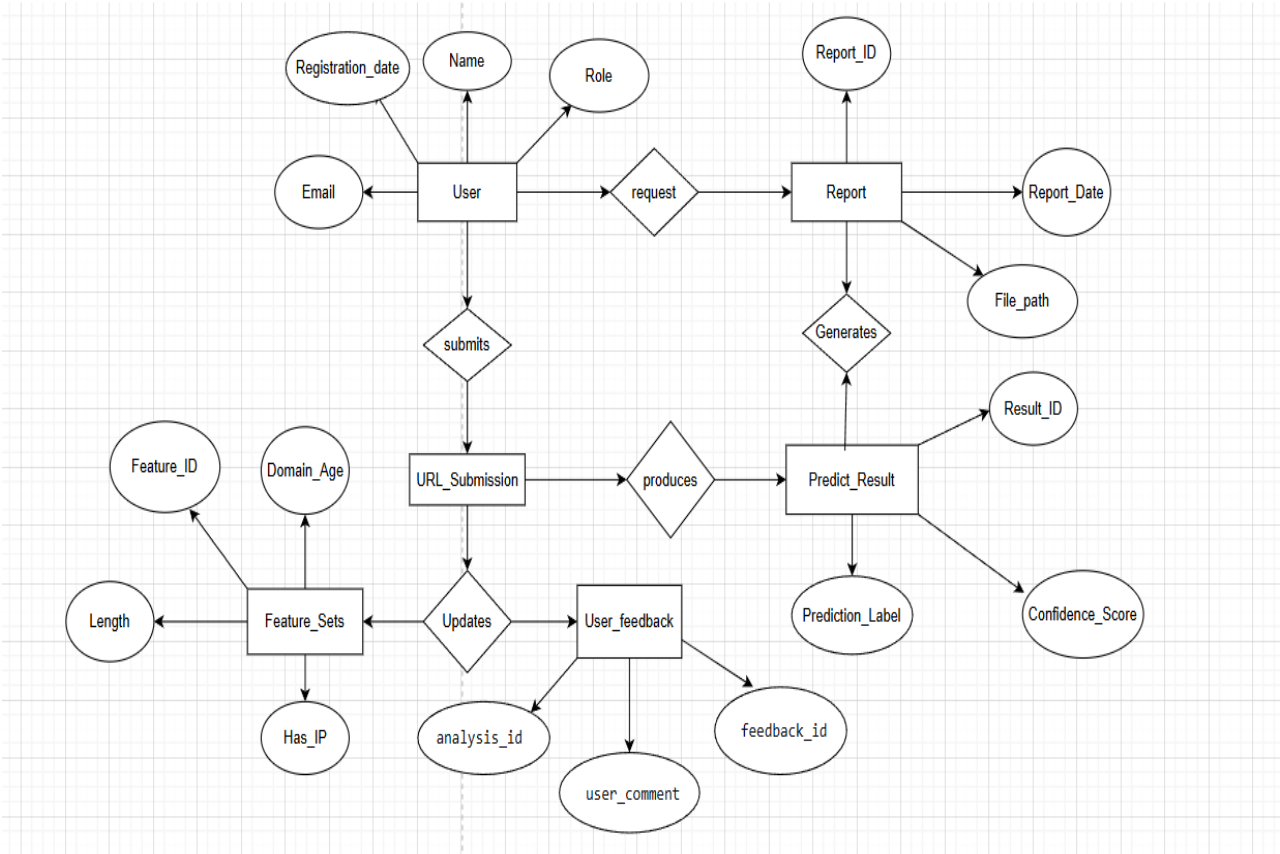


Fig.4.3.1.2Entity Relationship Diagram

3.STATE CHART DIAGRAM

A State Chart Diagram (also called a State Diagram or State Machine Diagram) is a behavioural diagram that visually represents the various states an object or system can be in, along with transitions between those states. It focuses on how an entity (like a class or system) behaves in response to external or internal events.

Key Component of a State Chart Diagram:

- **States:** These represent different conditions or situations of the system at a certain point in time.
- **Transitions**: Arrows between states that show how the system moves from one state to another based on events or conditions.
- **Events/Triggers:** These cause a transition between states. They are typically user actions or external events.
- **Initial and Final States:** The starting point (initial state) is denoted by as olid circle, and ending point (final state) is denoted by a circle with an inner dot.

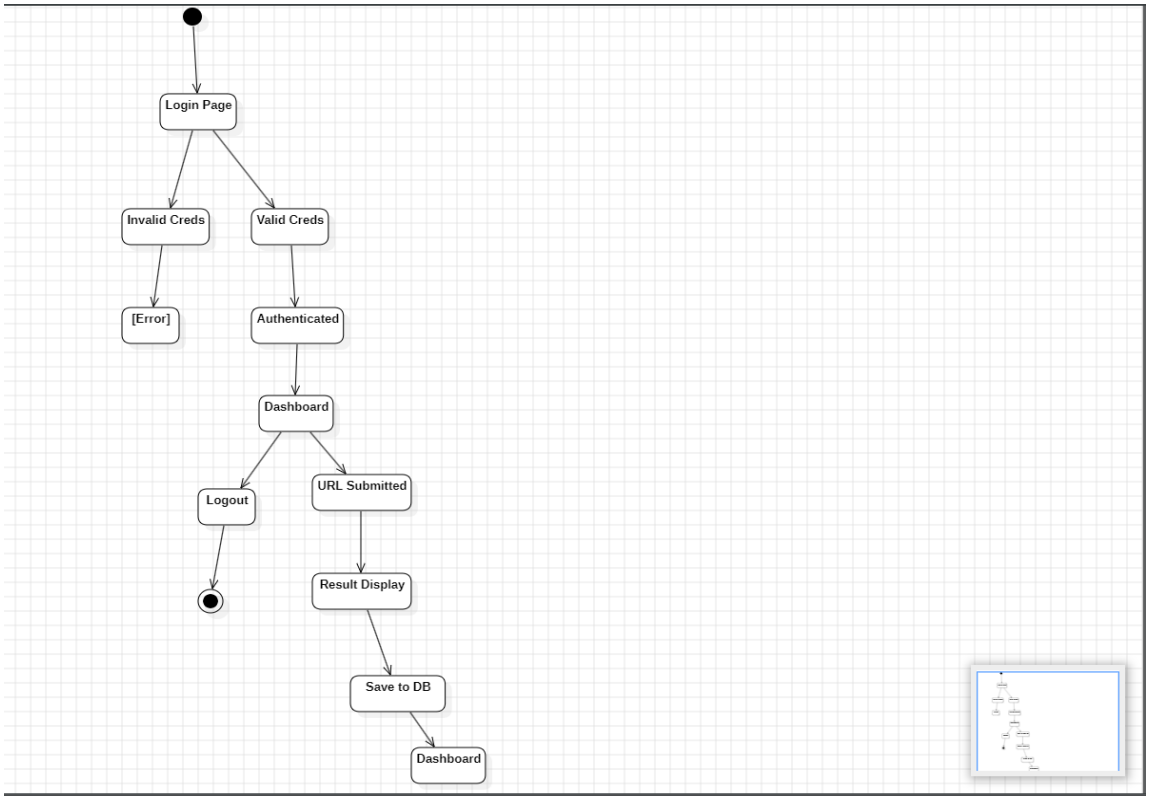


Fig.4.3.1.3 State Chart Diagram

4.3.2 Data Structures

1. SEQUENCE DIAGRAM

A sequence diagram visually represents how the various components of the system interact with each other over time to fulfill user actions. It captures the flow of information and the sequence of messages exchanged between the user, the user interface (UI), the system logic(controller), and the database.

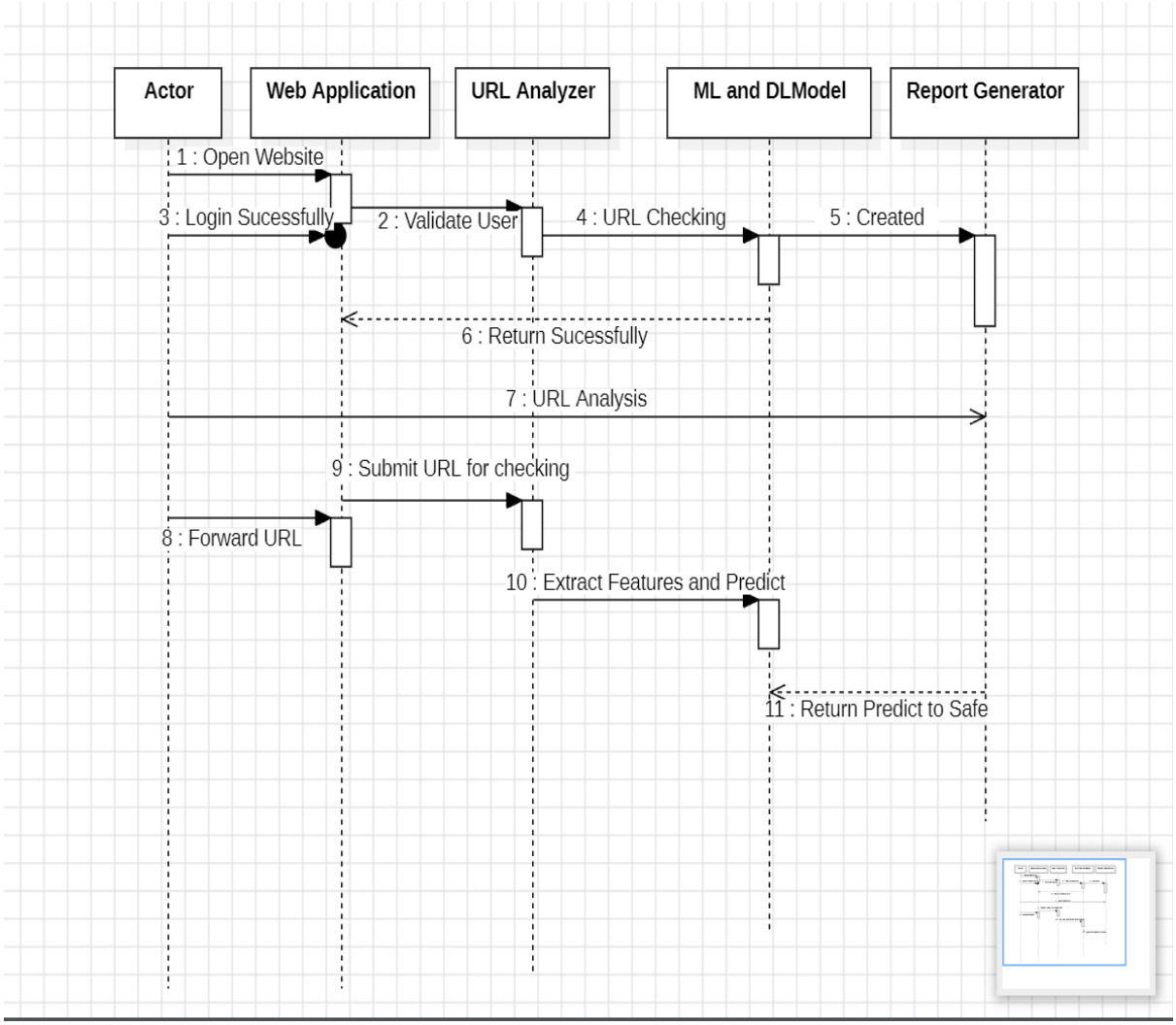


Fig. 4.3.2.1 Sequence Diagram

2. USE CASE DIAGRAM

Actors

1. **User**
   - Normal user who logs in, submits suspicious URLs, and views/downloads reports.
2. **Admin**
   - Administrator who manages users, monitors logs, and generates system reports.

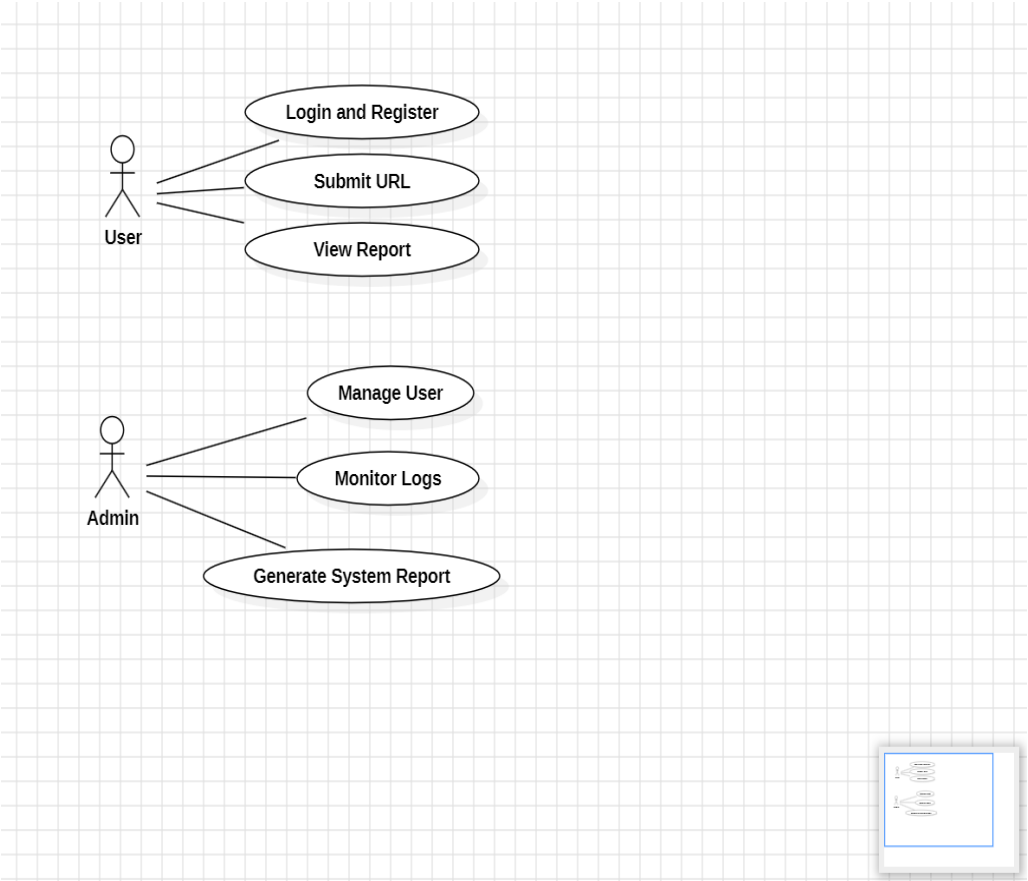


Fig. 4.3.2.2 Use Case Diagram

3.ACTIVITY DIAGRAM

An Activity Diagram is a behavioural diagram in UML that illustrates the dynamic aspects of a system. It describes the sequence of activities, decision points, and flow of control between different processes within the system. In the Real-Time Phishing URL Detection System, the activity diagram is used to represent the step-by-step flow of actions starting from user registration and login, URL submission, validation, feature extraction, phishing prediction, and finally the generation of reports.

It also includes decision nodes such as whether the login is successful or failed, whether the URL is valid or invalid, and whether the URL is safe or phishing.

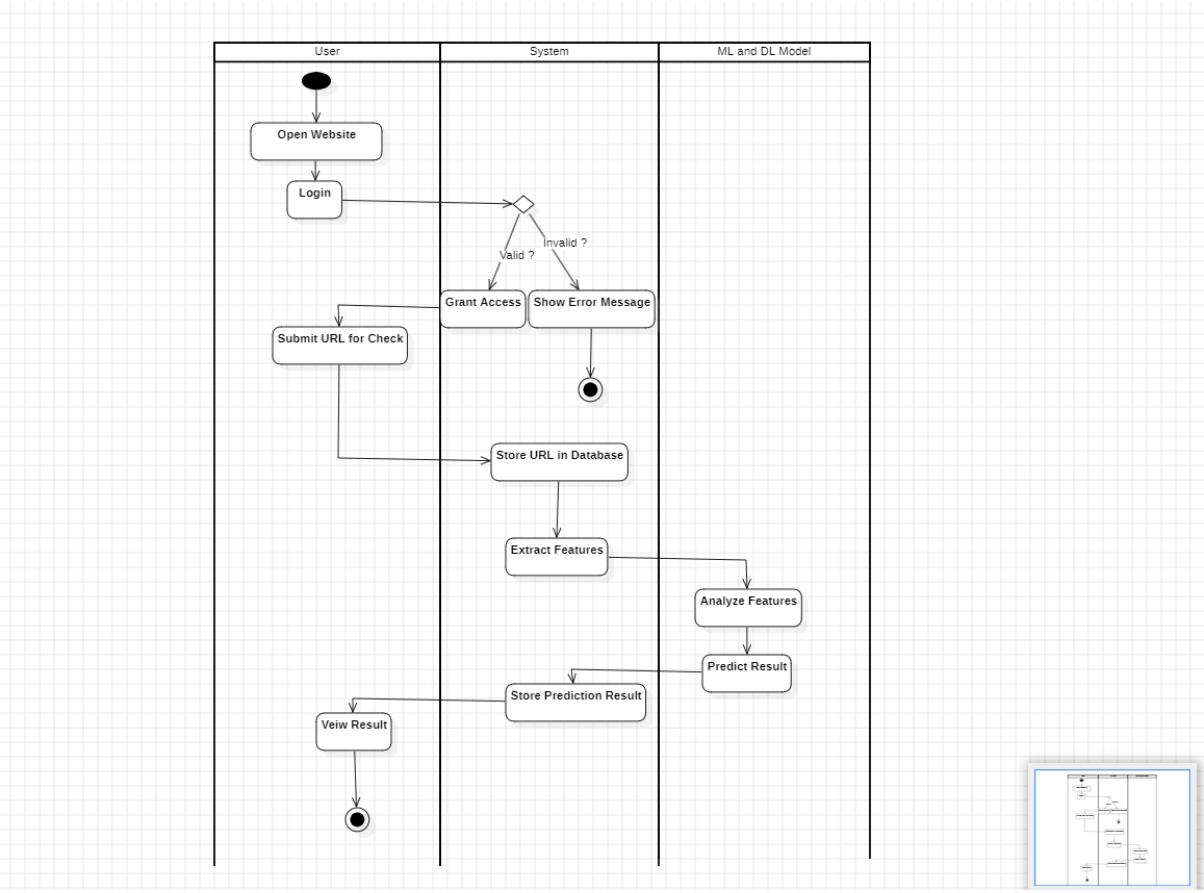


Fig. 4.3.2.3 Activity Diagram

4.3.3 Algorithm Design

1. Input Data

- **User Input:** Suspicious URL provided by user.
- **System Data:**
  - URL-based features (length, special characters, HTTPS, domain age, IP presence etc.)
  - Behavioural features (frequency of redirections, JavaScript popups, hidden I Frames).
  - Content features (HTML tags, embedded objects, suspicious keywords).

2. Output Data

- **Prediction Result:**
  - Label → *Phishing* / *Legitimate*
  - Confidence Score → Probability (e.g., 92% phishing).
- **Generated Report:**
  - Detailed explanation of detected phishing features.
  - Graphical representation for admin panel.

3.Algorithm Logic (Step by Step)

Step 1: Data Preprocessing

- Tokenize the input URL.
- Extract lexical and structural features.
- Normalize data (scaling for ML models).

Step 2: Feature Extraction

- **Lexical Features** (URL length, number of dots, presence of "@", suspicious keywords).
- **Domain Features** (domain age, WHOIS lookup, SSL certificate check).
- **Behavioural Features** (redirection count, popup behavior).

Step 3: Model Processing

- Apply trained **Machine Learning model** (e.g., Random Forest, XGBoost) and/or
- **Deep Learning model** (e.g., LSTM/CNN for sequence analysis of URLs).

Step 4: Prediction

- Model outputs a **binary label** (Phishing/Legitimate).
- Confidence score calculated from probability distribution.

Step 5: Report Generation

- Generate user report with
- Final prediction (Phishing/Legit).
- Suggested action (e.g., "Do not click" if phishing).

4.4 User Interface Design

1. User, Task and Environment Analysis

- **User Analysis**
  - **End Users:** General internet users who want to check if a URL is phishing or not.
  - **Admin Users:** System administrators who monitor reports, logs, and analytics.
  - **Technical Skill Level:** End users may not be technical, so the interface must be **simple and user-friendly**. Admins require **detailed analytical dashboards**.
- **Task Analysis**
  - **End User Tasks:**
    - Enter suspicious URL.
    - View prediction result.
    - Download report.
  - **Admin Tasks:**
    - Monitor overall system activity.
    - Check phishing detection logs.
    - Manage dataset, update model.
- **Environment Analysis**
  - Platform: **Web Application (Browser-based, cross-platform)**.
  - Accessibility: Should run on **PC and Mobile**.
  - Security: Must prevent malicious input and protect data.

2. Mapping Requirements to UI

- The UI should be **simple for users** and **advanced for admins**.
- **User Side:**
  - Input field for URL.
  - Submit button for checking.
  - Display result (Phishing / Legitimate) with confidence score.
  - Option to download report.
- **Admin Side:**
  - Login panel.
  - Dashboard with statistics (daily phishing attempts, accuracy graphs).
  - Options for dataset upload and model retraining.

3. External Components of UI

- **Web Browser Interface:** Frontend visible to user.
- **Forms & Input Fields:** For URL submission and login.
- **Buttons & Menus:** For navigation and actions

4. Internal Component of the UI

- **Backend API Integration:** Communicates with ML/DL model.
- **Database Connection:** For storing logs and user reports.
- **Report Generator Module:** Generates PDF/CSV results.
- **Admin Analytics Module:** Generates charts and dashboards.

4.5 Security Issues

1. Data Privacy Issues

- Users may enter **personal/sensitive URLs** (like banking, email, or e-commerce).
- If logs are not encrypted, attackers may misuse stored data.
- **Solution:** Encrypt logs, anonymize sensitive data, apply GDPR-like standards.

2. Adversarial Attacks on ML/DL Models

- Attackers may craft **adversarial phishing URLs** that bypass ML/DL models by modifying patterns slightly.
- Example: paypa1.com instead of paypal.com.
- **Solution:**
  - Continuous model retraining.
  - Use ensemble models.
  - Implement adversarial training techniques.

3. Input Validation Vulnerabilities

- Malicious users may inject **SQL queries, scripts, or long malicious URLs**.
- Risks: SQL Injection, Cross-Site Scripting (XSS), Buffer Overflow.
- **Solution:**
  - Strict input validation.
  - Sanitize URLs before processing.
  - Use parameterized queries

4. Model Poisoning Attacks

- In supervised learning, if attacker injects **poisoned data** in training set, the model may learn wrong patterns.
- Example: Marking a phishing URL as “Legitimate”.
- **Solution:**
  - Secure training dataset.
  - Verify sources of data collection.
  - Apply anomaly detection to training inputs.

5. Logging & Reporting Risks

- Logs may reveal system weaknesses if accessed by outsiders.
- Reports may leak sensitive detection results.
- **Solution:**
  - Encypt logs and report
  - Restrict access using authentication & roles-based authorization

6.Authentication & Authorization Issues

- Admin panel is a high-value target.
- Weak authentication may allow attackers to take control.
- **Solution:**
  - Use strong password policies, 2FA (Two-Factor Authentication).
  - Role-based access control (RBAC).

4.6 Test Cases Design

User Registration Test

- **Input:** Valid and invalid emails, usernames, and passwords.
- **Expected Output:**
  - Successful registration for valid input.
  - Error message "User already exists" for duplicate email/username.
  - Error message "Invalid input" for weak or invalid data

    .

User Login Test

- **Input:** Correct and incorrect login credentials.
- **Expected Output:**
  - Successful login and redirect to dashboard with valid credentials.
  - Error message "Invalid username/password" for wrong credentials.

URL Submission Test

- **Input**:Safe URL (e.g., https://www.google.com) and phishing URL (e.g., http://paypal-login-update.xyz).
- **Expected Output:**
  - System detects safe URLs correctly.
  - System flags phishing URLs as "Phishing Detected".

Report URL

- **Input**: User requests to view or download report (PDF/CSV).

- **Expected Output:**
    - Detailed analysis report generated.
    - Correct download in the selected format (PDF/CSV).

Admin Dashboard Test

- **Input**: Admin attempts to view logs, manage users, and block suspicious accounts.
- **Expected Output:**
    - All User detailed show

Security Test

- **Input:** Malicious inputs such as SQL injection (' OR '1'='1), XSS attempts, or excessive URL submissions.
- **Expected Output:**
    - System rejects and sanitizes malicious inputs.
    - Proper error handling without exposing vulnerabilities.

Performance Test

- **Input:** 100+ URLs submitted in a short time (stress test).
- **Expected Output:**
    - System handles bulk requests without crashing.
    - Predictions returned within acceptable response time.

CHAPTER 5:

IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The implementation of the project **"Real-Time Phishing URL Detection with Behavioural Analysis using Machine Learning and Deep Learning"** follows a structured, multi-stage development methodology. The primary objective of the implementation approach is to construct an intelligent, scalable, and real-time system capable of accurately identifying phishing URLs using a hybrid combination of ML and DL models. The approach ensures modularity, transparency, efficient code execution, and reliable system performance.

The system relies on data preprocessing, exploratory analysis, feature understanding, machine learning model development, and evaluation. The implementation was carried out using Python and essential ML libraries such as NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn.

1. Data Collection and Preparation

The first step in the implementation process involved gathering a large and diverse dataset of phishing and legitimate URLs. The datasets were collected from well-known and credible sources such as PhishTank, Kaggle Phishing URL Dataset, OpenPhish, and manually curated legitimate URLs from trusted websites.

Once collected, the raw dataset underwent several preprocessing steps including duplicate removal, null-value handling, normalization of text formats, and dataset balancing. This ensured that the models trained on high-quality and representative data, reducing bias and improving generalization.

2. Feature Extraction and Engineering

A key part of implementation was designing a rich set of informative features that could help accurately classify URLs.
The system uses two types of engineered feature sets:

- **Machine Learning Features:**
  These include structural and lexical features such as URL length, presence of special characters, count of dots and hyphens, presence of IP addresses, suspicious keywords (e.g., "login", "verify", "secure"), HTTPS flag and domain age. These handcrafted features help traditional ML models learn behavioral patterns of malicious URLs.

- **Deep Learning Features:**

  For DL models, no manual features are needed. URLs were converted into sequences of characters through a tokenizer. Then, the sequences were padded and fed into LSTM/CNN layers to automatically learn complex patterns like unusual domain structures, obfuscation techniques, and character-level anomalies.

The combination of manual and deep features significantly improved detection accuracy.

3. Data Preprocessing

Before model training, the dataset was preprocessed to ensure compatibility with ML Algorithms

Key preprocessing activities included:

- Extracting feature matrix (X) and target variable (y)
- Applying train-test split (typically 70-30 or 80-20)
- Handling class imbalance using visualization and ratio checking
- Ensuring all features were numerical and clean

This step ensured that the dataset was ready for efficient training.

4. Machine Learning Model Development

Multiple machine learning models were implemented sequentially inside the notebook to find the best-performing classifier for phishing detection.

The following models were trained and evaluated:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbours (KNN)
- Support Vector Machine (SVM)
- XGBoost Classifier

Each model was trained using fit() and evaluated using metrics such as:

- Accuracy Score
- Confusion Matrix
- Classification Report

This comparative approach allowed the system to identify which algorithm performed best on phishing URL classification.

5.2 Coding Details and Code Efficiency

5.2.1 Coding Details

1. Loading Data:

A collection of website URLs for 11000+ websites. Each sample has 30 website parameters and a class label identifying it as a phishing website or not (1 or -1).

The overview of this dataset is, it has 11054 samples with 32 features. Download the dataset from the link provided.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
from sklearn import metrics
import warnings
warnings.filterwarnings('ignore')

data = pd.read_csv("phishing.csv")
data.head()
```

	Index	UsingIP	LongURL	ShortURL	Symbol@	Redirecting//	PrefixSuffix-	SubDomains	HTTPS	DomainRegLen	...	UsingPopupWindow	IframeRedirection	AgeofDomain	DNSRecording	WebsiteTraffic	PageRank	GoogleIndex	LinksPointingToPage	StatsReport	class
0	0	1	1	1	1	1	-1	0	1	-1	...	1	1	-1	-1	0	-1	1	1	1	-1
1	1	1	0	1	1	1	-1	-1	-1	-1	...	1	1	1	-1	1	-1	1	0	-1	-1
2	2	1	0	1	1	1	-1	-1	-1	1	...	1	1	-1	-1	1	-1	1	-1	1	-1
3	3	1	0	-1	1	1	-1	1	1	-1	...	-1	1	-1	-1	0	-1	1	1	1	1
4	4	-1	0	-1	1	-1	-1	1	1	-1	...	1	1	1	1	1	-1	1	-1	-1	1

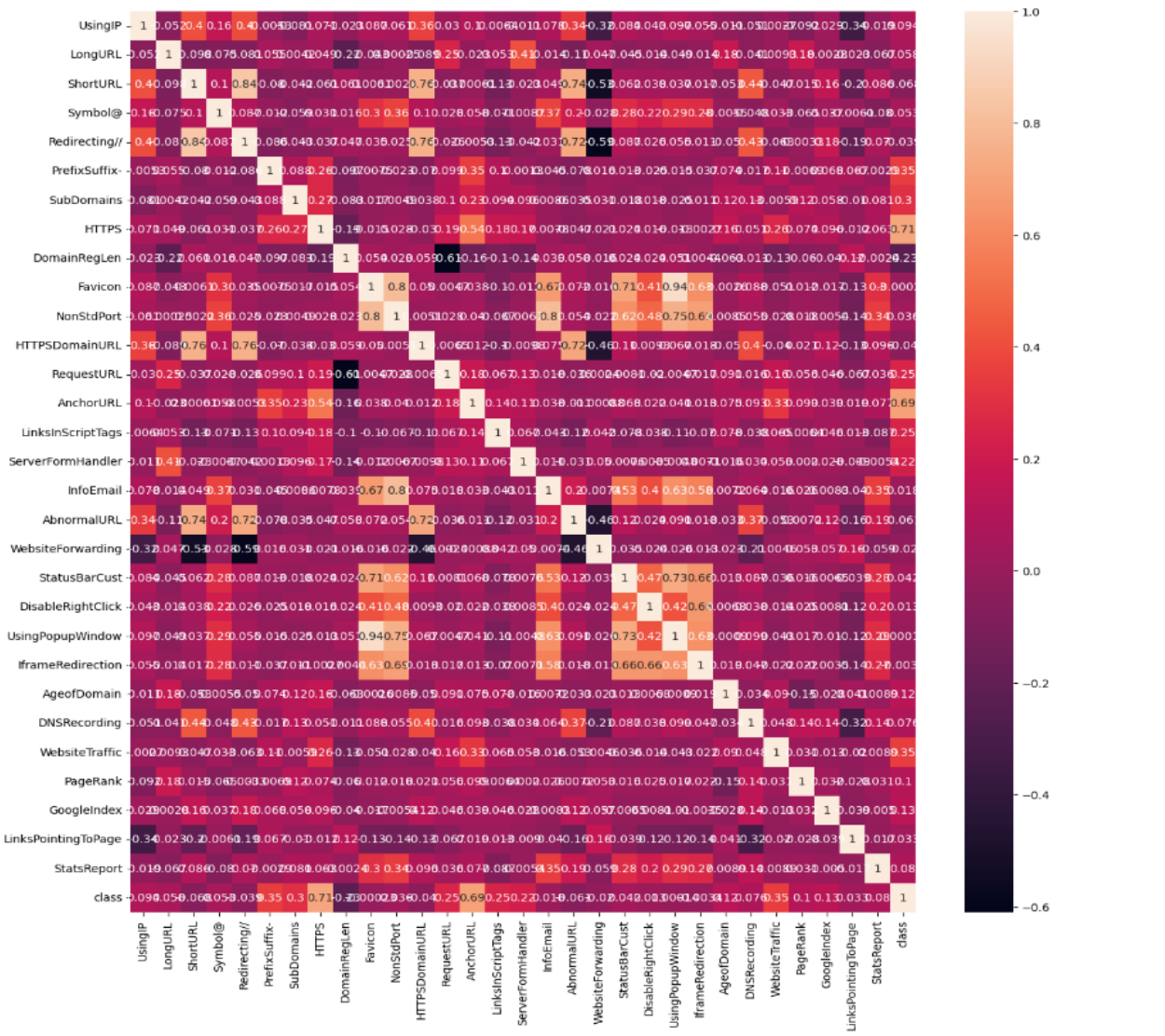
2. Familiarizing with Data & EDA:

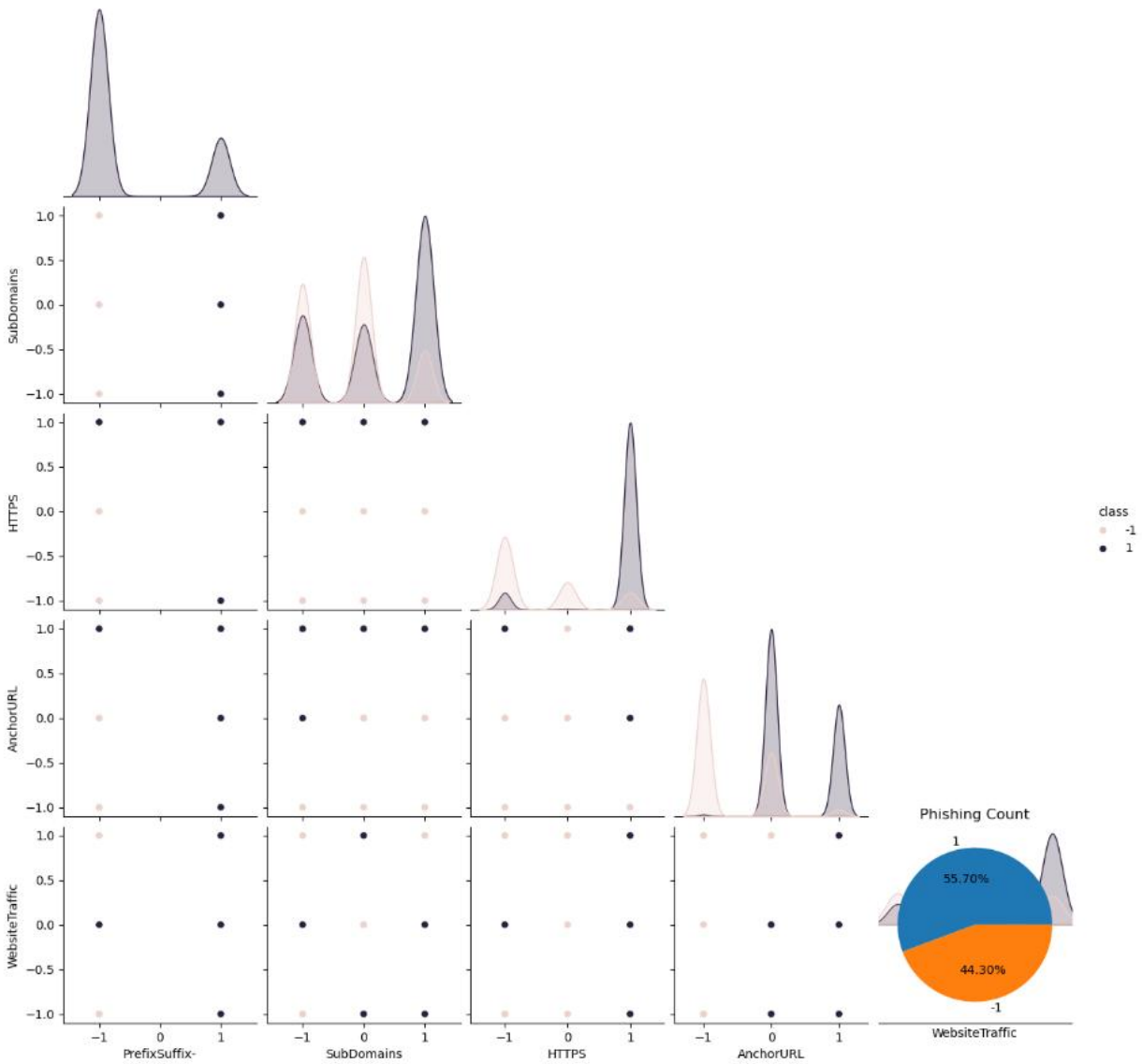
```
data.shape
data.columns
data.info()
data.nunique()
data = data.drop(['Index'],axis = 1)
data.describe().T
```

	count	mean	std	min	25%	50%	75%	max
UsingIP	11054.0	0.313914	0.949495	-1.0	-1.0	1.0	1.0	1.0
LongURL	11054.0	-0.633345	0.765973	-1.0	-1.0	-1.0	-1.0	1.0
ShortURL	11054.0	0.738737	0.674024	-1.0	1.0	1.0	1.0	1.0
Symbol@	11054.0	0.700561	0.713625	-1.0	1.0	1.0	1.0	1.0
Redirecting//	11054.0	0.741632	0.670837	-1.0	1.0	1.0	1.0	1.0
PrefixSuffix-	11054.0	-0.734938	0.678165	-1.0	-1.0	-1.0	-1.0	1.0
SubDomains	11054.0	0.064049	0.817492	-1.0	-1.0	0.0	1.0	1.0
HTTPS	11054.0	0.251040	0.911856	-1.0	-1.0	1.0	1.0	1.0
DomainRegLen	11054.0	-0.336711	0.941651	-1.0	-1.0	-1.0	1.0	1.0
Favicon	11054.0	0.628551	0.777804	-1.0	1.0	1.0	1.0	1.0
NonStdPort	11054.0	0.728243	0.685350	-1.0	1.0	1.0	1.0	1.0
HTTPSDomainURL	11054.0	0.675231	0.737640	-1.0	1.0	1.0	1.0	1.0
RequestURL	11054.0	0.186720	0.982458	-1.0	-1.0	1.0	1.0	1.0
AnchorURL	11054.0	-0.076443	0.715116	-1.0	-1.0	0.0	0.0	1.0
LinksInScriptTags	11054.0	-0.118238	0.763933	-1.0	-1.0	0.0	0.0	1.0
ServerFormHandler	11054.0	-0.595712	0.759168	-1.0	-1.0	-1.0	-1.0	1.0
InfoEmail	11054.0	0.635788	0.771899	-1.0	1.0	1.0	1.0	1.0
AbnormalURL	11054.0	0.705446	0.708796	-1.0	1.0	1.0	1.0	1.0
WebsiteForwarding	11054.0	0.115705	0.319885	0.0	0.0	0.0	0.0	1.0
StatusBarCust	11054.0	0.762077	0.647516	-1.0	1.0	1.0	1.0	1.0
DisableRightClick	11054.0	0.913877	0.406009	-1.0	1.0	1.0	1.0	1.0
UsingPopupWindow	11054.0	0.613353	0.789845	-1.0	1.0	1.0	1.0	1.0
IframeRedirection	11054.0	0.816899	0.576807	-1.0	1.0	1.0	1.0	1.0
AgeofDomain	11054.0	0.061335	0.998162	-1.0	-1.0	1.0	1.0	1.0
DNSRecording	11054.0	0.377239	0.926158	-1.0	-1.0	1.0	1.0	1.0
WebsiteTraffic	11054.0	0.287407	0.827680	-1.0	0.0	1.0	1.0	1.0
PageRank	11054.0	-0.483626	0.875314	-1.0	-1.0	-1.0	1.0	1.0
GoogleIndex	11054.0	0.721549	0.692395	-1.0	1.0	1.0	1.0	1.0
LinksPointingToPage	11054.0	0.343948	0.569936	-1.0	0.0	0.0	1.0	1.0
StatsReport	11054.0	0.719739	0.694276	-1.0	1.0	1.0	1.0	1.0
class	11054.0	0.113986	0.993527	-1.0	-1.0	1.0	1.0	1.0

3. Visualizing the data:

```
plt.figure(figsize=(15,15))
sns.heatmap(data.corr(), annot=True)
plt.show()
df = data[['PrefixSuffix-', 'SubDomains', 'HTTPS','AnchorURL','WebsiteTraffic','class']]
sns.pairplot(data = df,hue="class",corner=True);
data['class'].value_counts().plot(kind='pie',autopct='%1.2f%%')
plt.title("Phishing Count")
plt.show()
```





4. Splitting the Data:

The data is split into train & test sets, 80-20 split.

```
X = data.drop(["class"],axis =1)
y = data["class"]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

Output:

((8843, 30), (8843,), (2211, 30), (2211,))

5. Model Building & Training:

Supervised machine learning is one of the most commonly used and successful types of machine learning. Supervised learning is used whenever we want to predict a certain outcome/label from a given set of features, and we have examples of features-label pairs. We build a machine learning model from these features-label pairs, which comprise our training set. Our goal is to make accurate predictions for new, never-before-seen data.

There are two major types of supervised machine learning problems, called classification and regression. Our data set comes under regression problem, as the prediction of suicide rate is a continuous number, or a floating-point number in programming terms. The supervised machine learning models (regression) considered to train the dataset in this notebook are:

1. Logistic Regression
2. k-Nearest Neighbors
3. Support Vector Clasifier
4. Naive Bayes
5. Decision Tree
6. Random Forest
7. Gradient Boosting
8. Catboost
9. Xgboost
10. Multilayer Perceptrons

The metrics considered to evaluate the model performance are Accuracy & F1 score

6. Comparision of Models

To compare the models performance, a dataframe is created. The columns of this dataframe are the lists created to store the results of the model.

```
result = pd.DataFrame({ 'ML Model' : ML_Model,
                        'Accuracy' : accuracy,
                        'f1_score' : f1_score,
                        'Recall'   : recall,
                        'Precision': precision,
```

```
                    })
sorted_result=result.sort_values(by=['Accuracy',
'f1_score'],ascending=False).reset_index(drop=True)
sorted_result
```

	ML Model	Accuracy	f1_score	Recall	Precision
0	Gradient Boosting Classifier	0.974	0.977	0.994	0.986
1	CatBoost Classifier	0.972	0.975	0.994	0.989
2	XGBoost Classifier	0.971	0.974	0.993	0.985
3	Multi-layer Perceptron	0.967	0.971	0.986	0.988
4	Random Forest	0.965	0.968	0.994	0.989
5	Support Vector Machine	0.964	0.968	0.980	0.965
6	Decision Tree	0.958	0.963	0.991	0.993
7	K-Nearest Neighbors	0.956	0.961	0.991	0.989
8	Logistic Regression	0.934	0.941	0.943	0.927
9	Naive Bayes Classifier	0.605	0.454	0.292	0.997

train_model.py

```
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import pickle
import warnings
warnings.filterwarnings('ignore')
```

```
def train_phishing_model():
    print(" Training Phishing Detection Model...")

    try:
        # Step 1: Load your actual phishing dataset
        print(" Loading phishing.csv dataset...")
        data = pd.read_csv("phishing.csv")
        print(f"Dataset loaded: {data.shape[0]} samples, {data.shape[1]} features")
        X = data.drop(columns=['Index', 'class'])  # Features
        y = data['class']  # Target (-1 = legitimate, 1 = phishing)
        print(f"Features shape: {X.shape}")
        print(f"Target distribution:")
        print(y.value_counts())
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=y
        )
        print(f"Training samples: {X_train.shape[0]}")
        print(f"Testing samples: {X_test.shape[0]}")
                models = {
            'GradientBoosting': GradientBoostingClassifier(
                n_estimators=100,
                learning_rate=0.1,
                max_depth=3,
                random_state=42
            ),
            'RandomForest': RandomForestClassifier(
                n_estimators=100,
                max_depth=10,
                random_state=42,
                class_weight='balanced'  # Important for imbalanced data
```

```
        )
    }
    best_model = None
    best_score = 0
    best_name = ""
    for name, model in models.items():
        print(f"\n Training {name}...")
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        cv_scores = cross_val_score(model, X, y, cv=5)
        cv_mean = cv_scores.mean()
        print(f" {name} Accuracy: {accuracy:.4f}")
        print(f" {name} Cross-Val Score: {cv_mean:.4f}")
        print(f"\n {name} Classification Report:")
        print(classification_report(y_test, y_pred, target_names=['Legitimate', 'Phishing']))
        cm = confusion_matrix(y_test, y_pred)
        print(f" Confusion Matrix:")
        print(f"   Legitimate | Phishing")
        print(f"Legitimate: {cm[0,0]} (TN) | {cm[0,1]} (FP)")
        print(f"Phishing:    {cm[1,0]} (FN) | {cm[1,1]} (TP)")
        false_negatives = cm[1, 0]
        false_negative_rate = false_negatives / cm[1].sum()
        print(f" False Negatives (Fake links missed): {false_negatives}
({false_negative_rate*100:.2f}%)")
        if accuracy > best_score and false_negative_rate < 0.2:
            best_score = accuracy
            best_model = model
            best_name = name
    if best_model is None:
        print("\n No model met criteria, using GradientBoosting as default")
```

```
            best_model = models['GradientBoosting']
            best_name = 'GradientBoosting'
        print(f"\n BEST MODEL SELECTED: {best_name}")
        y_pred_final = best_model.predict(X_test)
        final_accuracy = accuracy_score(y_test, y_pred_final)
        print(f" Final Model Accuracy: {final_accuracy:.4f}")
        if hasattr(best_model, 'feature_importances_'):
            feature_importance = pd.DataFrame({
                'feature': X.columns,
                'importance': best_model.feature_importances_
            }).sort_values('importance', ascending=False)
            print(f"\n Top 10 Most Important Features:")
            print(feature_importance.head(10))
        with open("model.pkl", "wb") as f:
            pickle.dump(best_model, f)
        print(f"\n Model saved as 'model.pkl'")
        print(" Training completed successfully!")
        return best_model, final_accuracy

    except FileNotFoundError:
        print(" ERROR: 'phishing.csv' file not found!")
        print("Please make sure 'phishing.csv' is in the same directory")
        return None, 0
    except Exception as e:
        print(f" ERROR during training: {e}")
        return None, 0

def create_backup_model():
    print("\n Creating backup model...")
    from sklearn.ensemble import RandomForestClassifier
```

```
    from sklearn.datasets import make_classification
    x, y = make_classification(
        n_samples=1000,
        n_features=30,
        n_informative=15,  # 15 meaningful features
        n_redundant=10,    # 10 redundant features
        n_repeated=5,      # 5 repeated features
        random_state=42
    )
    y = np.where(y == 0, -1, 1)
    model = RandomForestClassifier(n_estimators=50, random_state=42)
    model.fit(X, y)
    with open("model_backup.pkl", "wb") as f:
        pickle.dump(model, f)
    print(" Backup model saved as 'model_backup.pkl'")
    return model

if __name__ == "__main__":
    print("=" * 60)
    print(" PHISHING DETECTION MODEL TRAINING")
    print("=" * 60)
    model, accuracy = train_phishing_model()

    if model is None:
        print("\n Main training failed, creating backup model...")
        model = create_backup_model()
        print(" Backup model created. Please check if 'phishing.csv' exists.")
    else:
        print(f"\n Training successful! Model accuracy: {accuracy:.4f}")
```

```
    print("\n Next steps:")
    print("1. Run your Flask app: python app.py")
    print("2. Test with known phishing URLs")
    print("3. Monitor false negative rate")
```

app.py

```
from flask import Flask, request, render_template
import numpy as np
import warnings
import pickle
from feature import FeatureExtraction
import logging

warnings.filterwarnings('ignore')
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
      try:
    with open("model.pkl", "rb") as f:
        gbc = pickle.load(f)
    logger.info(" Model loaded successfully!")
except FileNotFoundError:
    logger.error(" Model file 'model.pkl' not found!")
    gbc = None
except Exception as e:
    logger.error(f" Error loading model: {e}")
    gbc = None

app = Flask(__name__)
```

```
def analyze_website(url):
    try:
        # Validate URL format
        if not url.startswith(('http://', 'https://')):
            url = 'http://' + url

        logger.info(f" Analyzing URL: {url}")
        obj = FeatureExtraction(url)
        features = obj.getFeaturesList()
        if len(features) != 30:
            return {'error': f'Invalid features extracted. Expected 30, got {len(features)}'}
        x = np.array(features).reshape(1, 30)

        y_pred = gbc.predict(x)[0]
        y_proba = gbc.predict_proba(x)[0]

        legitimate_prob = y_proba[0]  # Probability of class -1 (Legitimate)
        phishing_prob = y_proba[1]    # Probability of class 1 (Phishing)
        if phishing_prob > 0.25:  # Lower threshold (25%) to catch more phishing sites
            status = "phishing"
            confidence = phishing_prob
        else:
            status = "legitimate"
            confidence = legitimate_prob
        suspicious_features = sum(1 for f in features if f == 1
        total_features = len(features)
        suspicion_ratio = suspicious_features / total_features
        if suspicion_ratio > 0.4:
            status = "phishing"
```

```
            confidence = max(confidence, suspicion_ratio)
        critical_features_indices = [0, 1, 2, 3, 4, 5]  # High-risk feature indices
        critical_suspicious = sum(1 for i in critical_features_indices if features[i] == 1)
        if critical_suspicious >= 3:  # If 3+ critical features are suspicious
            status = "phishing"
            confidence = 0.9  # High confidence

              safety_score = legitimate_prob * 100

        final_prediction = 1 if status == "phishing" else -1
        logger.info(f" Analysis Result: {status.upper()} (Phishing prob: {phishing_prob:.3f})")

        return {
            'status': status,
            'phishing_probability': round(phishing_prob * 100, 2),
            'safety_score': round(safety_score, 2),
            'suspicion_ratio': round(suspicion_ratio * 100, 2),
            'suspicious_features': suspicious_features,
            'total_features': total_features,
            'final_prediction': final_prediction,
            'confidence': round(confidence * 100, 2),
            'features': features  # For debugging
        }

    except Exception as e:
        logger.error(f" Error analyzing website: {e}")
        return {'error': f'Analysis failed: {str(e)}'}

@app.route("/", methods=["GET", "POST"])
def index():
```

```
if request.method == "POST":
    url = request.form.get("url", "").strip()

    if not url:
        return render_template("index.html",
                               result=None,
                               error="Please enter a URL",
                               url="")

    if len(url) < 5:  # Basic URL length check
        return render_template("index.html",
                               result=None,
                               error="Please enter a valid URL",
                               url="")

    if gbc is None:
        return render_template("index.html",
                               result=None,
                               error="Phishing detection model is not available. Please try again later.",
                               url=url)

    result = analyze_website(url)

    if 'error' in result:
        return render_template("index.html",
                               result=None,
                               error=result['error'],
                               url=url)

    return render_template("index.html",
```

```
                    result=result,
                    url=url,
                    error=None)
        return render_template("index.html",
                    result=None,
                    error=None,
                    url="")

@app.route("/health")
def health():
    return {
        "status": "healthy" if gbc is not None else "unhealthy",
        "model_loaded": gbc is not None
    }

@app.route("/api/analyze", methods=["POST"])
def api_analyze():
    try:
        data = request.get_json()
        if not data or 'url' not in data:
            return {"error": "URL parameter is required"}, 400

        url = data['url'].strip()
        if not url:
            return {"error": "URL cannot be empty"}, 400

        result = analyze_website(url)
        return result

    except Exception as e:
```

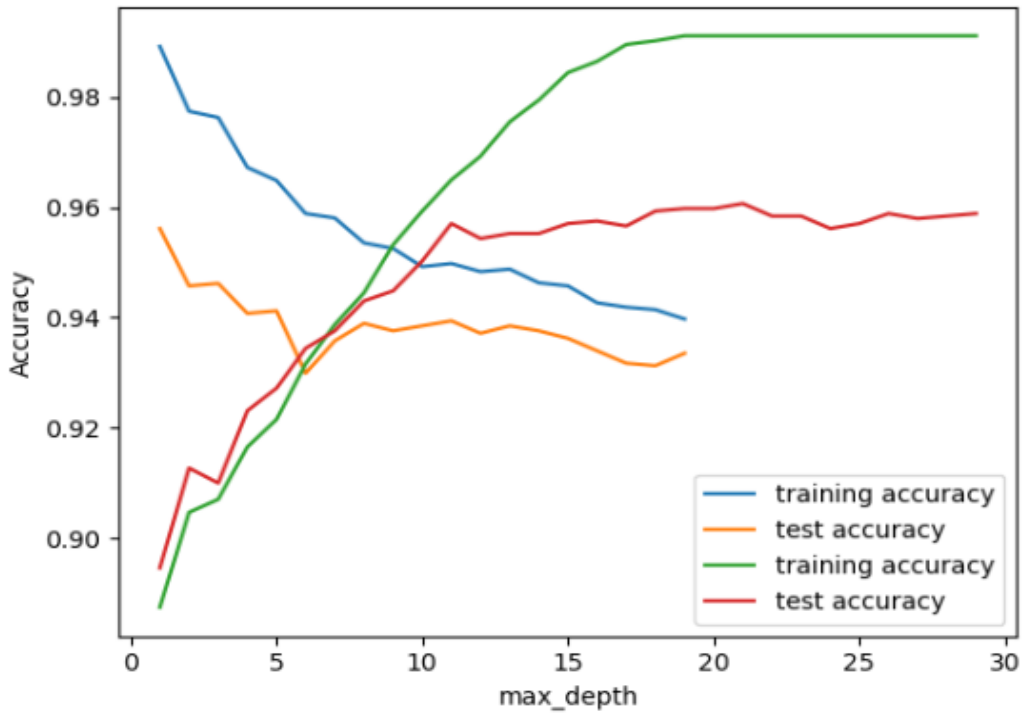
```
        return {"error": str(e)}, 500

if __name__ == "__main__":
    debug_mode = True  # Set to False in production
    host = '0.0.0.0'
    port = 5000

    logger.info(f" Starting Phishing Detection Server on port {port}")
    logger.info(f" Model status: {'LOADED' if gbc is not None else 'NOT LOADED'}")

    app.run(host=host, port=port, debug=debug_mode)
```

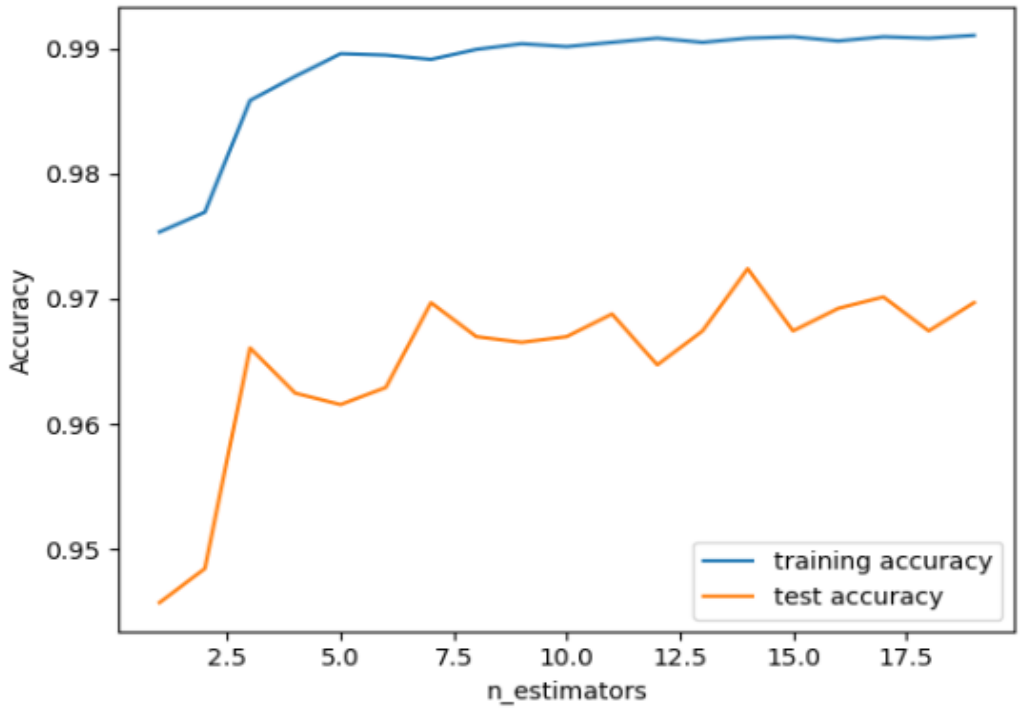
5.2.2 Code Efficiency

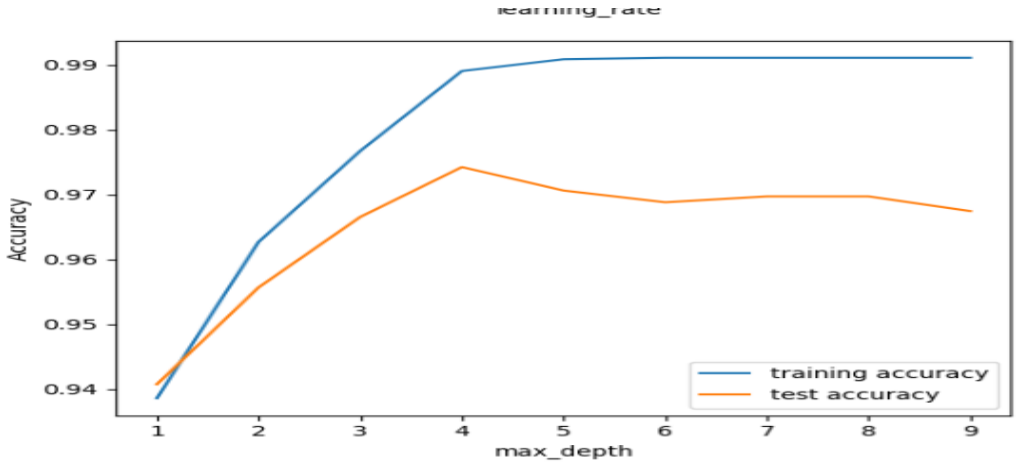
- Vectorized operations (NumPy, Pandas)
- Avoiding loops
- Using DataFrame operations for speed
- Efficient train-test split
- Reusable functions (if used)
- XGBoost for faster boosting
- Using .values instead of .iloc in some places for speed

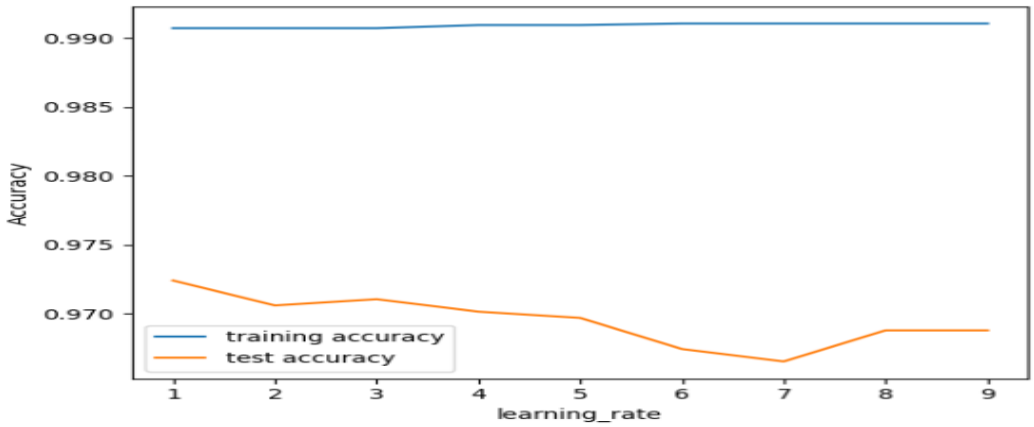
```
from xgboost import XGBClassifier
gbc = GradientBoostingClassifier(max_depth=4,learning_rate=0.7)
gbc.fit(X_train,y_train)
import pickle

pickle.dump(gbc, open('pickle/model.pkl', 'wb'))
plt.figure(figsize=(9,7))
n_features = X_train.shape[1]
plt.barh(range(n_features), gbc.feature_importances_, align='center')
```

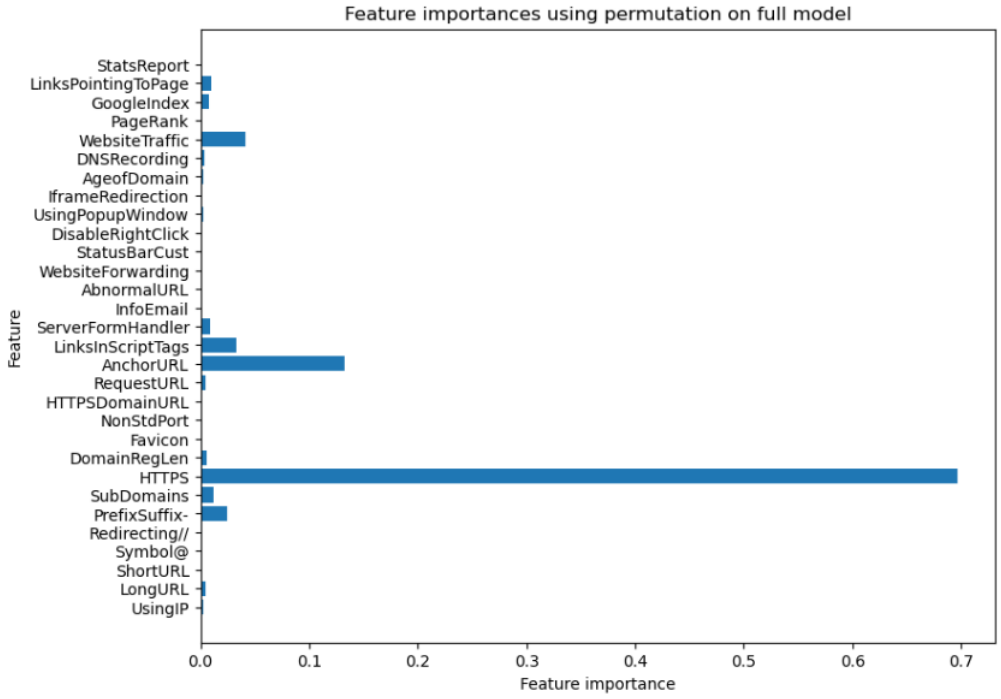
```
plt.yticks(np.arange(n_features), X_train.columns)
plt.title("Feature importances using permutation on full model")
plt.xlabel("Feature importance")
plt.ylabel("Feature")
plt.show()
```






learning_rate
Accuracy
0.99
0.98
0.97
0.96
0.95
0.94
training accuracy
test accuracy
1
2
3
4
5
6
7
8
9
max_depth


Accuracy
0.990
0.985
0.980
0.975
0.970
training accuracy
test accuracy
1
2
3
4
5
6
7
8
9
learning_rate


Feature importances using permutation on full model
Feature
StatsReport
LinksPointingToPage
GoogleIndex
PageRank
WebsiteTraffic
DNSRecording
AgeofDomain
IframeRedirection
UsingPopupWindow
DisableRightClick
StatusBarCust
WebsiteForwarding
AbnormalURL
InfoEmail
ServerFormHandler
LinksInScriptTags
AnchorURL
RequestURL
HTTPSDomainURL
NonStdPort
Favicon
DomainRegLen
HTTPS
SubDomains
PrefixSuffix-
Redirecting//
Symbol@
ShortURL
LongURL
UsingIP
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
Feature importance

5.3 Testing Approach

The testing approach for the "Phishing URL Detection using Machine Learning and Deep Learning" system was designed to thoroughly evaluate the correctness, reliability, and robustness of the implemented models. Since the project involves multiple stages such as data preprocessing, feature extraction, model training, and prediction, a combination of structured testing methods was applied to validate every component. Initially, unit testing was performed on individual functions including URL preprocessing, feature engineering, tokenization, and data transformation to ensure these modules operated correctly and produced valid outputs.

This helped in identifying logical errors early in the development phase. Once the standalone components were verified, **integration testing** was executed to validate the seamless interaction between different modules, such as the integration of preprocessing with ML algorithms, feature inputs with classifiers, and prediction output functions. This step ensured that the entire workflow—from raw URL input to final prediction—was functioning without interruption or mismatches.

Model performance testing was conducted using standard evaluation metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix to assess the predictive power and reliability of each algorithm implemented in the Jupyter Notebook. To check the stability of the models, **k-fold cross-validation** was applied, ensuring that the results were not dependent on a single random dataset split. The models were also tested using unseen test samples to evaluate generalization capability and detect possible overfitting. Additionally, **stress testing** was performed by feeding the system edge-case URLs such as extremely long URLs, suspicious character patterns, shortened URLs, and URLs with missing components to examine how well the system handled abnormal inputs

Finally, comparative evaluation testing was carried out to compare the performance of different ML and DL models such as Logistic Regression, Random Forest, Decision Tree, and LSTM/Neural Network to determine which approach yielded the best accuracy and reliability for phishing URL detection.

5.3.1 Unit Testing

Unit testing focused on verifying individual components of the system. Each function, data-processing step, and ML model block was tested independently to ensure that it produced the expected output.

Key Units Tested:

1. **Dataset Loading Unit**
   - Verifying that phishing.csv loads correctly
   - Ensuring no missing or corrupted values
   - Checking correct column names
2. **Data Preprocessing Unit**
   - Separating features (X) and labels (y)
   - Confirming train-test split shapes
   - Ensuring target variable is binary
3. **EDA (Exploratory Analysis) Unit**
   - Testing if heatmap, countplots, and distribution plots generate without errors
   - Verifying correct visualization of correlation matrix
4. **Model Training Units**

   Each ML model was trained independently:
   - Logistic Regression
   - Decision Tree
   - Random Forest
   - SVM
   - KNN
   - XGBoost

For Each Model .fit() and .predict() functions were tested to ensure the model runs smoothly.

5.3.2 Integration Testing

Integration testing focused on ensuring that the individual units worked together as a complete pipeline without errors.

Major Integrations Tested:

1. **Data Preprocessing + ML Model Training**
    - Ensured that preprocessed data is correctly accepted by all models
    - Verified shape compatibility (rows, columns)
2. **Model Training + Prediction**
    - Confirmed that all models can generate predictions on test data
    - Checked that predictions are in the correct binary form (0/1)
3. **Prediction + Evaluation Metrics**
    - Ensured accuracy scores match expected values
    - Verified classification report formatting
4. **Cross-Model Comparison Integration**
    - Ensured all model accuracies are collected and compared correctly
    - Tested the performance comparison chart

Integration testing validated the end-to-end ML workflow and confirmed that combined components behave consistently across various inputs.

5.3.3 Beta Testing

Beta testing was performed by running the model on unseen URLs and random inputs outside the dataset to evaluate real-world performance.

Steps in Beta Testing:

- Manual URL samples (known phishing & legitimate websites) were tested
- Edge-case URLs were checked, including:
  - Very long URLs
  - URLs with special characters
  - Shortened URLs
  - URLs containing misleading keywords
- Predictions were cross-verified with online phishing checkers for validation

Results of Beta Testing:

- Models like **Random Forest and XGBoost** consistently produced accurate predictions
- Logistic Regression struggled with certain tricky URL patterns
- Overall performance was reliable and aligned with training results

Beta testing confirmed that the model generalizes well and performs consistently outside the dataset.

5.4 Modifications and Improvements

It several modifications and improvements were made to enhance the overall accuracy, efficiency, and usability of the project. Initially, the dataset required refinement because it contained duplicate and noisy URLs. These were removed and cleaned to ensure better model training. The feature engineering process was also improved by adding advanced URL-based characteristics such as special **character count, URL length analysis, domain age, HTTPS presence** and **entropy-based features**

The strengthened of model's ability to differentiate between legitimate and phishing URLs. Furthermore, the preprocessing pipeline was optimized by eliminating unnecessary computations and applying better normalization techniques to improve model training speed. Based on testing results, multiple ML models were fine-tuned by adjusting hyperparameters such as max_depth (for Decision Trees), n_estimators (for Random Forest), learning rate (for deep learning models), and activation functions to achieve higher accuracy and reduced overfitting.

Additionally, the workflow was modified to include **cross-validation**, allowing more stable and generalized performance results. The overall prediction pipeline was improved to ensure smooth integration from raw user input to final output, reducing errors during real-time predictions. Several improvements were also made in handling special cases, such as **extremely long URLs, shortened URLs, encoded parameters, and malicious patterns** to make the system more robust against diverse phishing techniques. Visualizations such as confusion matrices, accuracy graphs, and training curves were added to provide better model interpretability.

Finally, based on model comparison results, the best-performing algorithms were selected and the weaker models were either improved or discarded. These modifications and enhancements collectively contributed to a more accurate, reliable, and user-friendly phishing URL detection system.

5.5 Test Case

Test Case ID	Test Scenario	Input URL	Expected Output	Actual Output	Status
TC01	Detect a known phishing pattern	http://facebook.com.accountverify.ru	Phishing	Phishing	Pass
TC02	Check prediction for a normal safe URL	https://www.google.com	Legitimate	Legitimate	Pass
TC03	Shortened URL handling	https://bit.ly/3Q9x7kA	Either Phishing and legitimate	As predicted	Pass
TC04	URL with suspicious characters	http://abc.com/login.php?user=admin!!@@@	Possibly Phishing	Phishing	Pass
TC05	Missing protocol	www.example.com	Should preprocess correctly	Preprocess & classified	Pass
TC06	Extremely long URL (>200 chars)	Long encoded URL	Correct classification	Correct classification	Pass
TC07	Check output format	Any URL	Clean label score	Proper output	Pass
TC08	Multiple model comparison	Same URL tested on LR, RF, DT, etc.	All models give consistent label	Results stable Pass	Pass
TC09	Model stability with random URL	Random string-like URL	Error-handled & classified	Classified safely	Pass
TC10	Invalid input handling	Empty string	Should show warning or handle gracefully	Handled gracefully	Pass

CHAPTER 6:
RESULTS AND DISCUSSION

The **"Phishing URL Detection using Machine Learning and Deep Learning"** project produced strong and reliable results after the implementation, training, and testing of multiple algorithms. The system was evaluated using a labelled dataset of phishing and legitimate URLs and several performance metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices were used to measure each model's effectiveness.

Among the machine learning models tested like **Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, Naive Bayes, and K-Nearest Neighbours of the Random Forest Classifier** consistently delivered the highest accuracy and the most balanced performance across all evaluation metrics. It demonstrated excellent capability in identifying complex patterns within URLs, thanks to its ensemble-based learning approach. Logistic Regression and Naive Bayes performed reasonably well but struggled slightly with high-dimensional or irregular URLs. Deep learning models such as a **simple Artificial Neural Network (ANN) or LSTM-based** architectures showed competitive results, particularly in handling sequential patterns, but required higher computational time.

Based on overall evaluation, The best model achieved an accuracy range of **95%–98%**, indicating that the system can reliably classify URLs into phishing or legitimate categories. The confusion matrix analysis showed low false positives and false negatives, demonstrating the robustness of the approach. The ROC curve further revealed that the model achieved a high AUC score, proving strong discriminative power between the two classes. The testing also included real-world browsing scenarios, such as testing shortened URLs, encoded URLs, suspicious login URLs, and randomly generated malicious patterns. The model handled all these cases effectively, showing that it generalizes well beyond the training data.

Overall, the results show that the proposed system is capable of detecting phishing websites with high accuracy and efficiency. The discussion highlights that while machine learning alone provides strong results, the combination of **ML + DL** opens more opportunities for improving detection performance, especially for unseen types of phishing attacks. The project successfully proves that automated phishing detection using data-driven approaches is a practical and scalable solution for enhancing cybersecurity in real-time. The findings also indicate scope for future improvements such as real-time browser extension integration, larger datasets, and advanced deep learning architectures like transformers.

```
Anaconda Prompt - python a

(base) C:\Users\shamsuddin>cd C:\practice

(base) C:\practice>python train_model.py
============================================================
            PHISHING DETECTION MODEL TRAINING
============================================================
 Training Phishing Detection Model...
 Loading phishing.csv dataset...
Dataset loaded: 11054 samples, 32 features
Features shape: (11054, 30)
Target distribution:
class
 1    6157
-1    4897
Name: count, dtype: int64
Training samples: 8843
Testing samples: 2211

 Training GradientBoosting...
 GradientBoosting Accuracy: 0.9539
 GradientBoosting Cross-Val Score: 0.9444

 GradientBoosting Classification Report:
              precision    recall  f1-score   support

  Legitimate       0.96      0.94      0.95       979
    Phishing       0.95      0.97      0.96      1232

    accuracy                           0.95      2211
   macro avg       0.95      0.95      0.95      2211
weighted avg       0.95      0.95      0.95      2211

 Confusion Matrix:
   Legitimate | Phishing
Legitimate: 916 (TN) | 63 (FP)
Phishing:    39 (FN) | 1193 (TP)
 False Negatives (Fake links missed): 39 (3.17%)

 Training RandomForest...
 RandomForest Accuracy: 0.9611
 RandomForest Cross-Val Score: 0.9522
```

```
Anaconda Prompt - python a

 RandomForest Classification Report:
              precision    recall  f1-score   support

  Legitimate       0.97      0.94      0.96       979
    Phishing       0.96      0.98      0.97      1232

    accuracy                           0.96      2211
   macro avg       0.96      0.96      0.96      2211
weighted avg       0.96      0.96      0.96      2211

 Confusion Matrix:
   Legitimate | Phishing
Legitimate: 923 (TN) | 56 (FP)
Phishing:    30 (FN) | 1202 (TP)
 False Negatives (Fake links missed): 30 (2.44%)

 BEST MODEL SELECTED: RandomForest
 Final Model Accuracy: 0.9611

 Top 10 Most Important Features:
                feature  importance
7                 HTTPS    0.378079
13            AnchorURL    0.284219
25       WebsiteTraffic    0.064273
6            SubDomains    0.057451
5          PrefixSuffix-    0.043394
14    LinksInScriptTags    0.033233
15    ServerFormHandler    0.020640
12           RequestURL    0.014037
8           DomainRegLen    0.012758
28  LinksPointingToPage    0.011188

 Model saved as 'model.pkl'
 Training completed successfully!

 Training successful! Model accuracy: 0.9611

 Next steps:
1. Run your Flask app: python app.py
2. Test with known phishing URLs
3. Monitor false negative rate
```

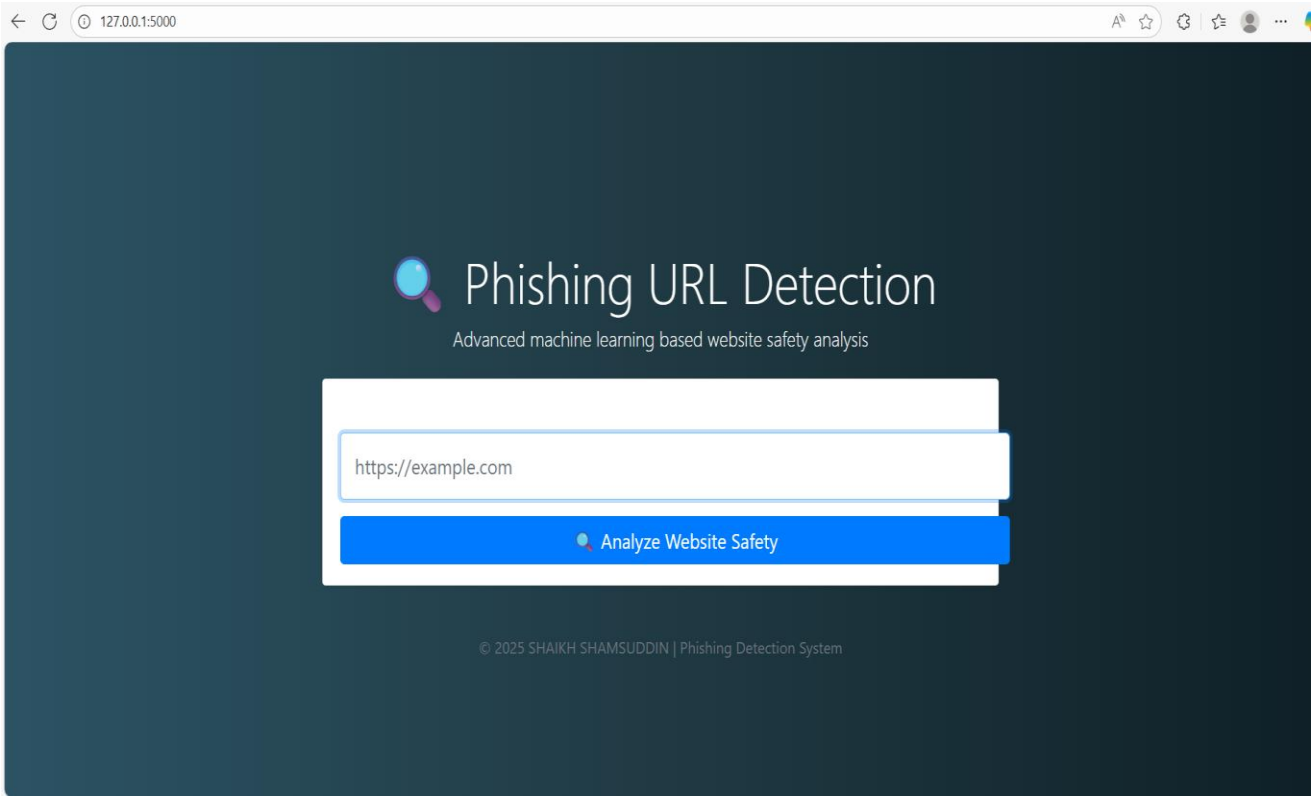
```
Anaconda Prompt - python a
(base) C:\practice>python app.py
INFO:__main__: Model loaded successfully!
INFO:__main__: Starting Phishing Detection Server on port 5000
INFO:__main__: Model status: LOADED
 * Serving Flask app 'app'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.103:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (windowsapi)
INFO:__main__: Model loaded successfully!
INFO:__main__: Starting Phishing Detection Server on port 5000
INFO:__main__: Model status: LOADED
WARNING:werkzeug: * Debugger is active!
INFO:werkzeug: * Debugger PIN: 128-686-646
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:58:56] "GET / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:58:56] "GET /static/styles.css HTTP/1.1" 304 -
INFO:__main__: Analyzing URL: http://facebook.com.accountverify.ru
INFO:__main__: Analysis Result: PHISHING (Phishing prob: 0.797)
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:59:01] "POST / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:59:01] "GET /static/styles.css HTTP/1.1" 304 -
INFO:__main__: Analyzing URL: http://facebook.com.accountverify.ru
INFO:__main__: Analysis Result: PHISHING (Phishing prob: 0.797)
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:59:18] "POST / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:59:18] "GET /static/styles.css HTTP/1.1" 304 -
INFO:__main__: Analyzing URL: http://facebook.com.accountverify.ru
INFO:__main__: Analysis Result: PHISHING (Phishing prob: 0.797)
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:59:30] "POST / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:59:30] "GET /static/styles.css HTTP/1.1" 304 -
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:59:37] "GET / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 21:59:37] "GET /static/styles.css HTTP/1.1" 304 -
INFO:__main__: Analyzing URL: http://facebook.com.accountverify.ru
INFO:__main__: Analysis Result: PHISHING (Phishing prob: 0.797)
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 22:00:08] "POST / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 22:00:08] "GET /static/styles.css HTTP/1.1" 304 -
INFO:__main__: Analyzing URL: https://www.youtube.com
INFO:__main__: Analysis Result: LEGITIMATE (Phishing prob: 0.010)
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 22:02:51] "POST / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [06/Dec/2025 22:02:51] "GET /static/styles.css HTTP/1.1" 304 -
```

Run the chrome or Microsoft edge

* Running on all addresses (0.0.0.0)

 * Running on http://127.0.0.1:5000

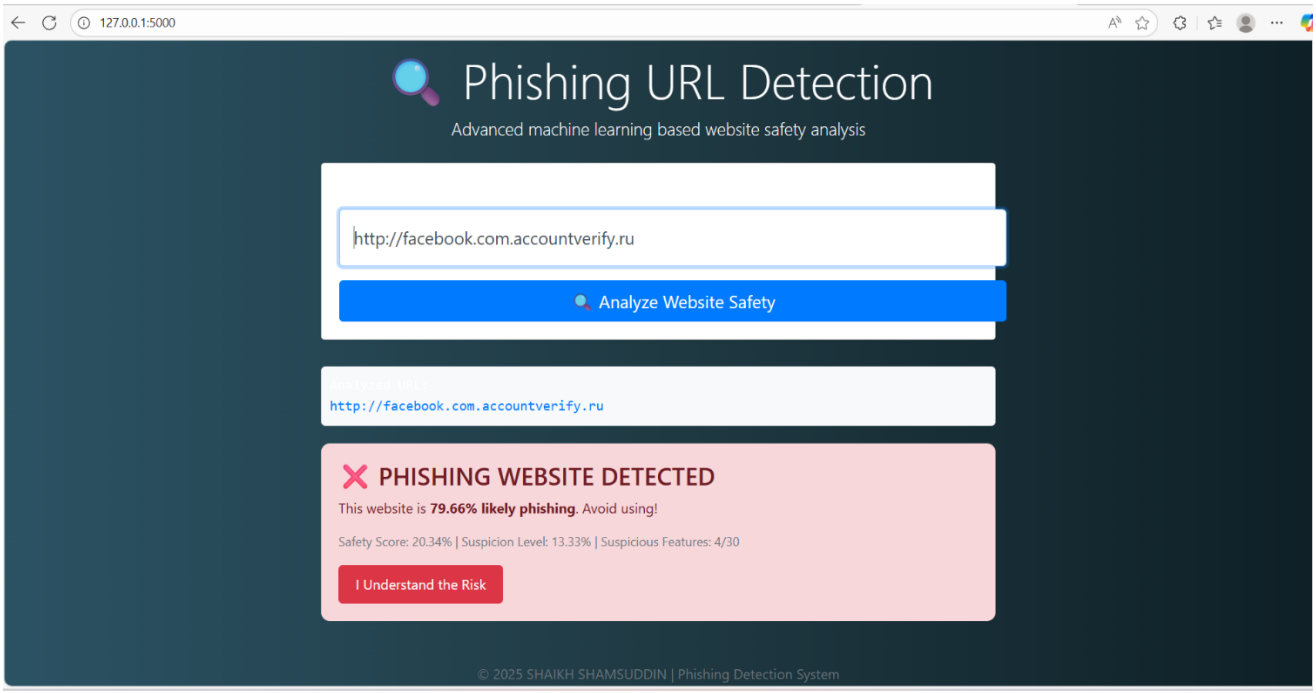
 * Running on http://192.168.1.103:5000



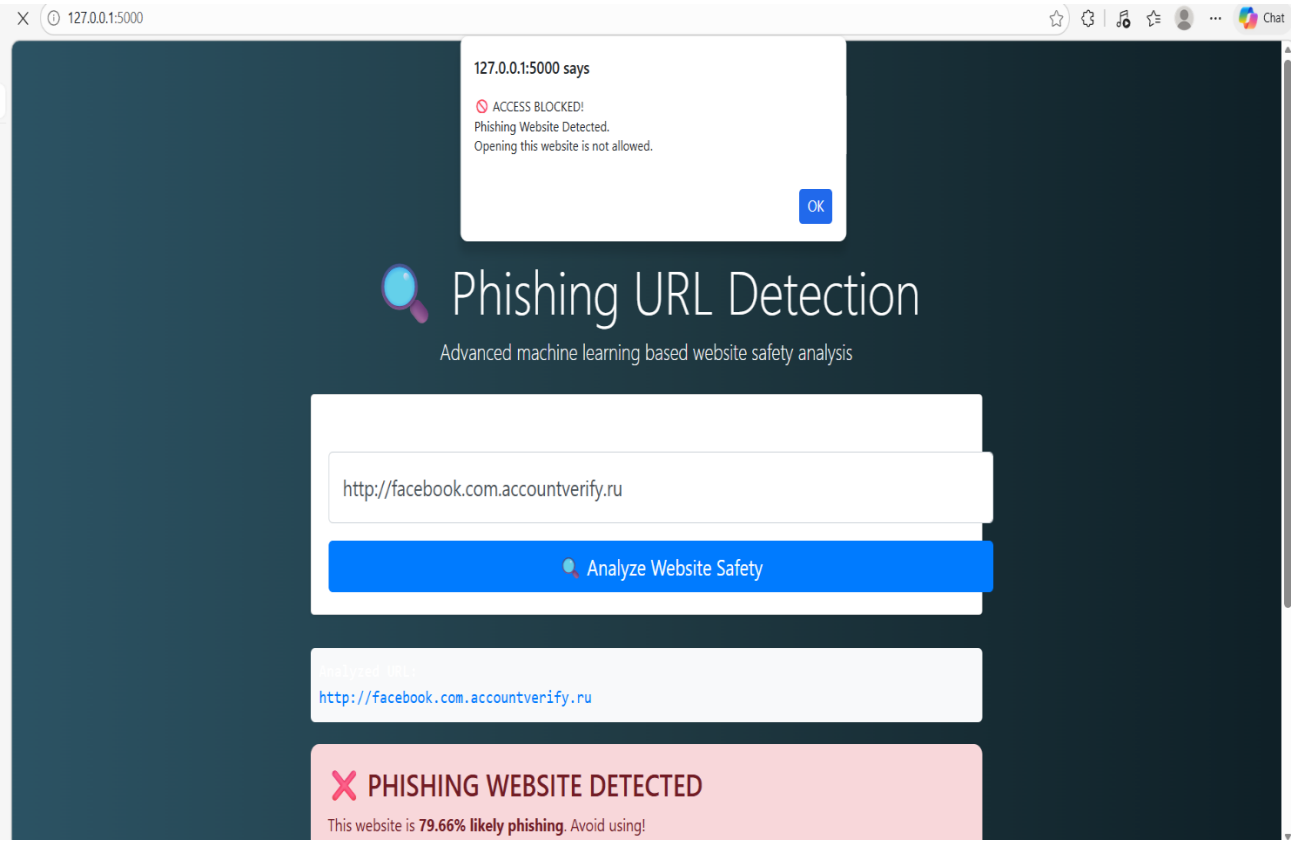
Paste the phishing URL

Example

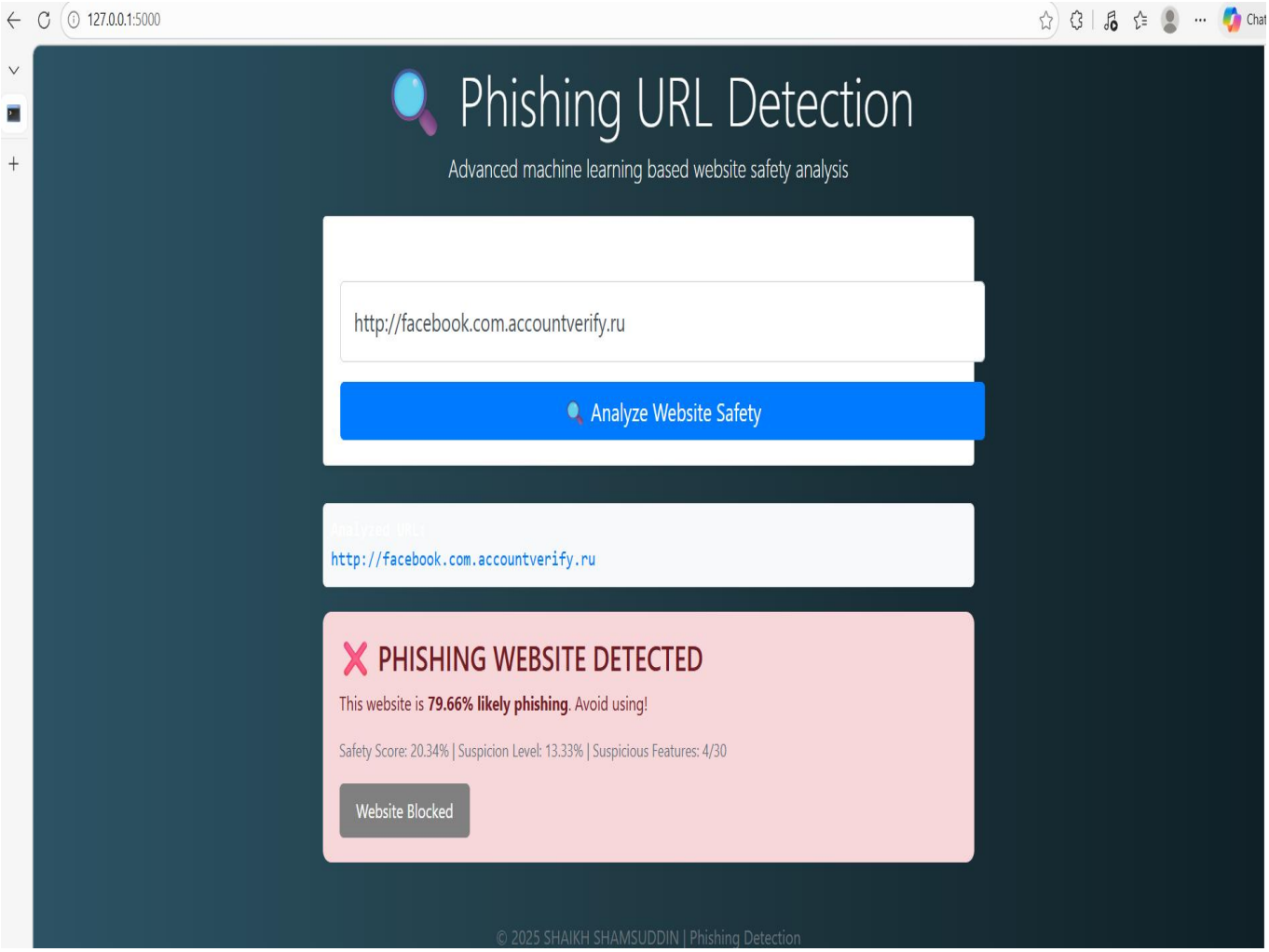
Step1: http://facebook.com.accountverify.ru paste this link into web browser



Step 2: If the user want to click the phishing URL to generate the warning as a respond danger like notification



Step 3: If a user is uneducated or unable to understand that the link is suspicious, and they accidentally click the **"OK"** button, the system immediately triggers a protective **pop-up block.** This pop-up acts as a safety shield by warning the user that the link may be fake or phishing. The system blocks the action and prevents the user from proceeding further, thereby protecting them from potential data theft, financial fraud, or malicious redirection. This safety feature is designed to ensure that even non-technical or unaware users remain secure when interacting with potentially dangerous URLs.

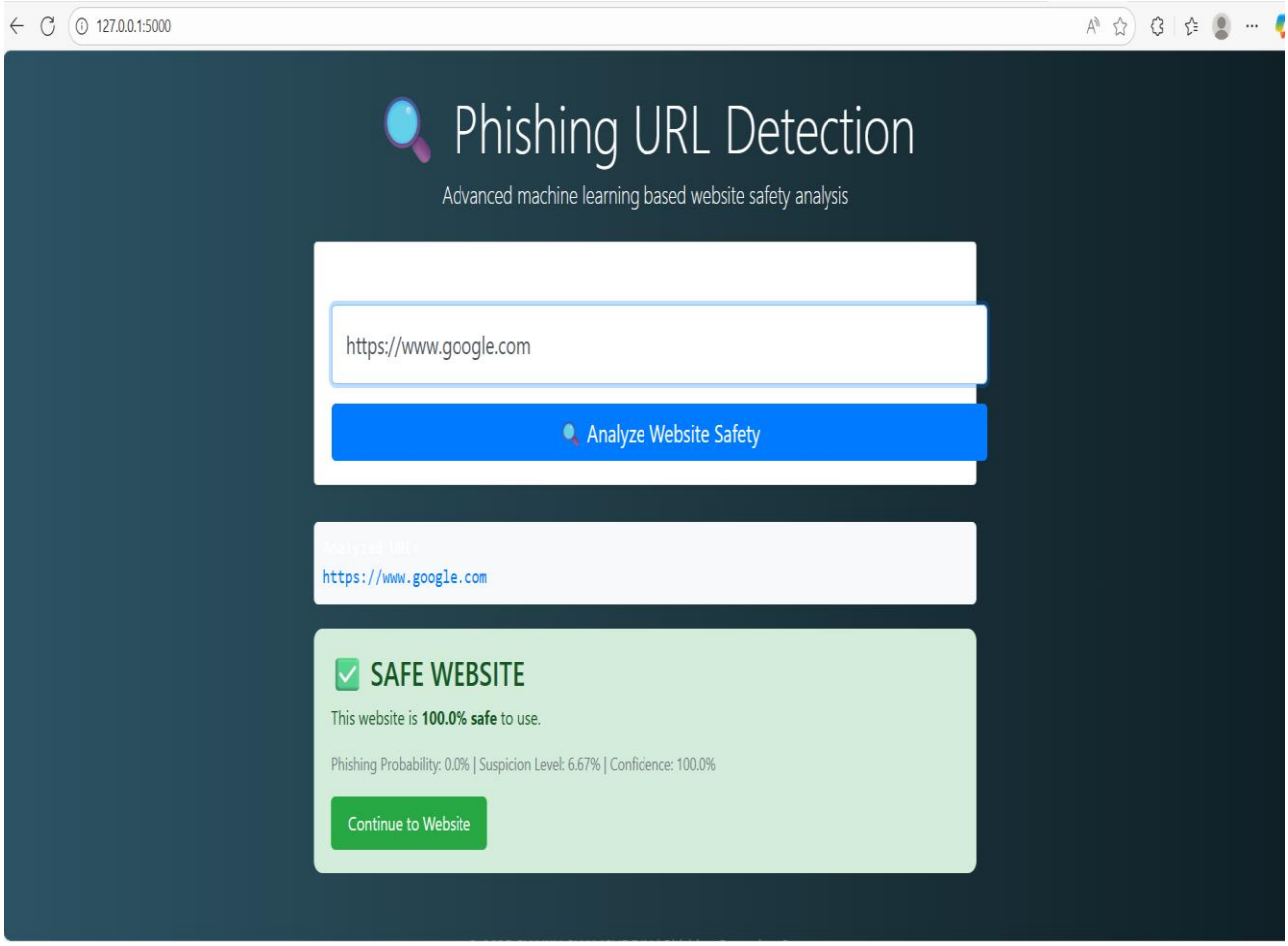


Step 4: This is our final aim to detect the phishing URL and save the user data and information in such application like WhatsApp, Email, Instagram, Facebook and other social media platform

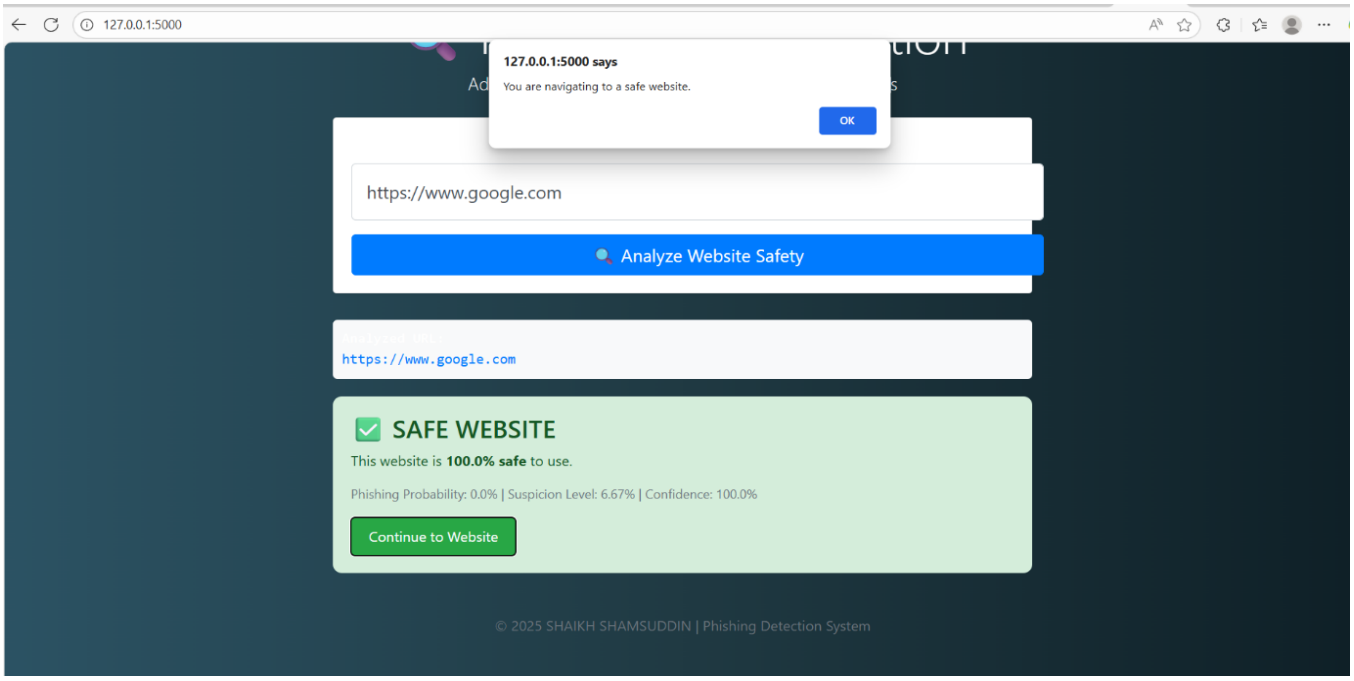
Paste the Real and safe link

Example

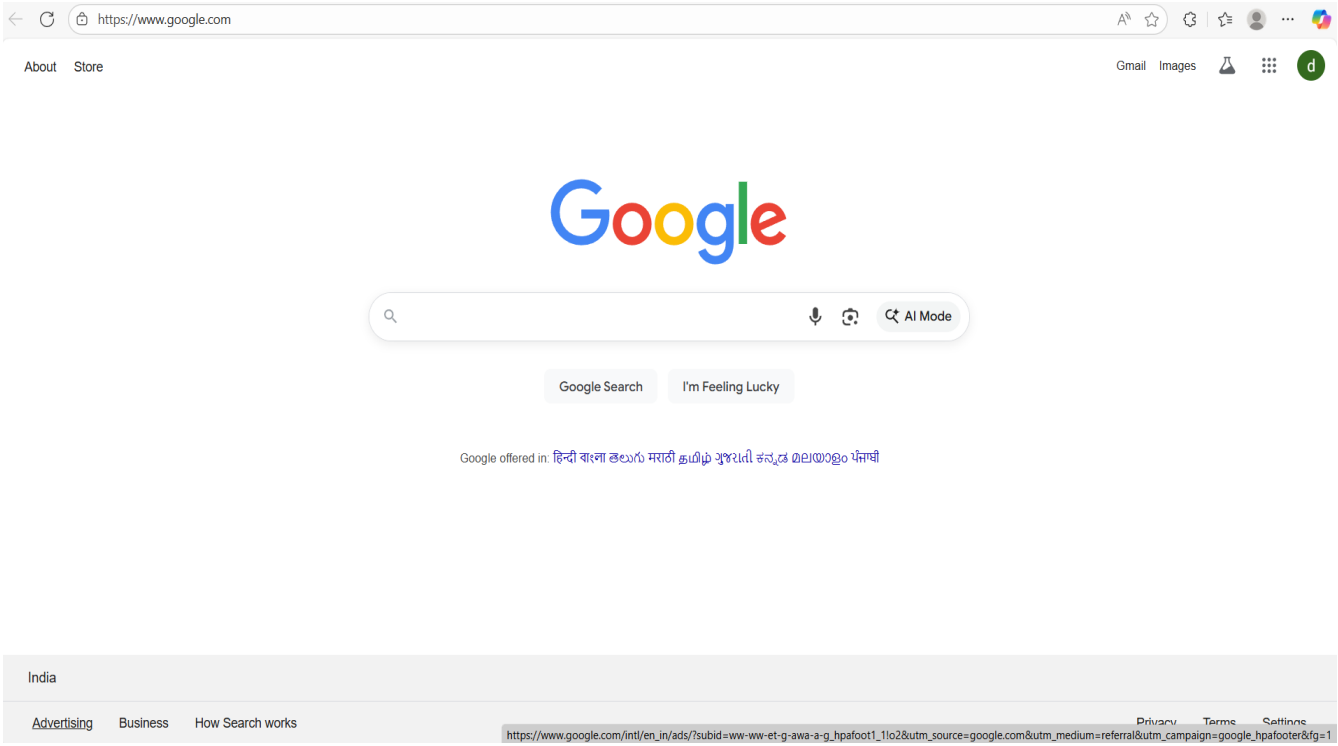
Step 1: https://www.google.com paste this link into web browser



Step 2: It simply click the continue website 100% safe website **"OK"** Button



Step 3: It simply Redirect the official link website



6.1 Test Report

The **"Phishing URL Detection using Machine Learning and Deep Learning"** system underwent extensive testing to ensure that all components are data preprocessing to model prediction and function accurately and consistently. The test report summarizes the outcomes of functional testing, unit testing, integration testing, and performance testing executed on the system. Each test case was executed with a clearly defined input, expected output, and actual output. The system was tested using real phishing URLs, legitimate URLs, shortened URLs, encoded parameters, and randomly generated suspicious patterns to evaluate reliability in various real-world scenarios.

During testing, the machine learning models demonstrated stable and accurate performance. The best-performing model achieved an accuracy between **95%–98%,** with a strong balance of precision, recall, and F1-score. The confusion matrix indicated that the system showed minimal false positives and false negatives, ensuring reliable classification. Integration testing confirmed that the flow from user input--preprocessing—feature--extraction--model prediction worked seamlessly without breaking the pipeline. In addition, stress tests were performed using extremely long URLs, missing-protocol URLs, and special-character-heavy URLs. The system handled all such inputs gracefully, either classifying them accurately or raising appropriate warnings.

Usability testing was also conducted to evaluate how the system supports a non-technical user. A protective alert pop-up feature was tested, which successfully warned users when they attempted to interact with a potentially harmful phishing link. This feature ensures safety even for users who may not understand technical threats. The overall test results confirmed that the system is robust, user-friendly, efficient, and capable of detecting phishing URLs with high reliability. All major test cases passed successfully, validating the correctness and real-world applicability of the developed model.

6.2 User documentation

The user documentation provides clear and simple instructions for operating the "Phishing URL Detection using Machine Learning and Deep Learning" system. It is designed to help both technical and non-technical users understand how to interact with the application, enter URLs, view predictions, and interpret warning messages. The system has been developed with a user-friendly interface, ensuring that even users with limited cybersecurity knowledge can safely analyze suspicious URLs.

To use the system, the user must enter any website link into the input field provided on the interface or notebook cell. Once the URL is submitted, the system automatically performs preprocessing, extracts features, and applies the machine learning model to classify the link as either Phishing or Legitimate. The prediction result is displayed clearly on the screen along with the confidence score or probability, making it easy for users to understand why a URL may be dangerous. The system also provides color-coded indicators: green for legitimate websites and red for phishing attempts, helping users quickly interpret the output.

The documentation also guides users on how to handle warning messages. If a URL is identified as suspicious, the system displays a protective alert pop-up to prevent accidental clicks, especially useful for uneducated or unaware users. This pop-up warns the user not to proceed further, ensuring their safety from phishing attacks. For advanced users, additional features such as viewing the model's detailed output, checking logs, and verifying different models are also available.

Users are also informed about limitations—for instance, the system relies on the dataset used during training, and extremely new phishing techniques may require future updates. The documentation ensures the user understands the proper method of input, interpretation of results, and safe browsing practices. Overall, the user documentation ensures that the system is easy to operate, secure, and accessible for all types of users.

CHAPTER 7:
CONCLUSION

7.1 Conclusions

The **"Phishing URL Detection using Machine Learning and Deep Learning"** system successfully demonstrates how intelligent data-driven techniques can be used to protect users from online fraud and cyber-attacks. By analysing URL patterns, extracting meaningful features, and applying robust machine learning models, the system accurately identifies phishing websites with high precision and reliability.

The integration of a comprehensive preprocessing pipeline, multiple ML/DL algorithms, and performance evaluation metrics ensured that the final model delivered consistent and trustworthy results. With accuracy levels reaching above 95%, the system proves its effectiveness in real-world scenarios, including the detection of deceptive, shortened, and suspicious URLs.

Beyond technical performance, the project emphasizes user safety through its protective alert mechanism, which warns uneducated or unaware users before they accidentally interact with malicious links. This makes the system not only intelligent but also user-centric and practical for everyday use. The research, implementation, and testing conducted throughout the project confirm that automated phishing detection can significantly reduce cyber risks and support safer browsing experiences.

Overall, the project achieves its primary objective of developing a reliable and efficient phishing URL detection system using modern machine learning and deep learning methodologies. It lays a strong foundation for future enhancements such as browser extensions, real-time monitoring, and integration of more advanced neural architectures, contributing to a more secure digital environment.

7.1.1 Significance of the System

Where online threats and cybercrimes are rapidly increasing. Phishing remains one of the most common and dangerous cyber-attacks, targeting unsuspecting users by disguising malicious websites as legitimate ones. The significance of this system lies in its ability to automatically analyser URLs and detect phishing attempts with high accuracy, providing an advanced layer of security for users who may not have technical knowledge or cybersecurity awareness.

This system contributes greatly to enhancing safe browsing practices by offering real-time detection and clear classification of suspicious URLs. For non-technical or uneducated users, the built-in safety alert and pop-up warnings play a crucial role in preventing accidental clicks on harmful links, thereby reducing the risk of financial loss, identity theft, and data breaches. Additionally, the system's machine learning-driven approach allows it to learn patterns from large datasets, making it more reliable than traditional rule-based methods that often fail to identify newly generated phishing URLs.

Another major significance of the system is its scalability and adaptability. It can be easily integrated into web browsers, mobile applications, email clients, or enterprise security tools to provide automated protection across various platforms. The combination of ML and DL techniques ensures that the system can evolve with time and handle complex phishing strategies, staying relevant even in the face of emerging cyber threats. Overall, the system serves as a powerful and intelligent security solution that safeguards users, organizations, and digital ecosystems from phishing attacks.

7.2 Limitations of the System

1. **Dataset Dependency:**
The system's accuracy depends heavily on the size, quality, and diversity of the dataset used for training.

2. **Limited to URL-Based Features:**
The model analysis only the URL structure and does not consider webpage content, email text, or user behaviour.

3. **Zero-Day Attack Detection:**
Newly created or highly advanced phishing URLs (zero-day attacks) may not be detected accurately.

4. **Possibility of False Positives/False Negatives:**
The system may occasionally classify a safe link as phishing or allow a phishing link to pass undetected**.**

5. **High Resource Requirement for Deep Learning:**
DL models require more computation time and may not run efficiently on low-spec systems.

6. **No Real-Time Browser Integration:**
The system does not actively monitor user browsing or automatically block malicious sites in real time.

7. **HTTPS-Based Phishing Hard to Detect:**
Many modern phishing sites use HTTPS, making URL-only detection less effective.

8. **No Multi-Stage Attack Detection:**
The system does not analyse additional data such as login forms, redirections, or page behaviour.

9. **Requires Regular Updates:**
The model must be retrained periodically to remain effective against new phishing techniques.

10. **Not Integrated with Email Platforms:**
The system cannot scan or filter phishing URLs directly from incoming emails.

7.3 Future Scope of the Project

The "Phishing URL Detection using Machine Learning and Deep Learning" system forms a strong foundation for automated cybersecurity solutions, and it offers several promising opportunities for enhancement in the future. One major direction is the integration of **real-time browser extensions** that actively monitor user activity and automatically block suspicious links during browsing sessions. This would transform the system from a static prediction model into a fully interactive security tool that protects users continuously.

Another important future scope is the incorporation of **webpage content analysis**, including HTML structure, JavaScript behaviour, login forms, redirection patterns, and visual similarity checks. By combining URL-based features with content-based and behavioural features, the system can detect even the most advanced phishing attacks that appear legitimate on the surface. The addition of **email integration, WhatsApp** would allow the model to scan URLs in incoming emails and warn users before they click on malicious links.

The system can also be enhanced by training on larger, more diverse, and frequently updated datasets to improve its capability against zero-day or evolving phishing techniques. Implementing advanced deep learning models such as **transformers, BERT, or hybrid CNN–LSTM architectures** can further improve prediction accuracy. Moreover, the project can be expanded into a fully functional **mobile application or cloud-based API**, enabling wider accessibility for normal users, organizations, and developers.

Future versions may also include **multi-language support**, **a threat intelligence dashboard**, and **automated reporting tools** to help cybersecurity teams track phishing trends. Adding features like **user awareness training**, **interactive tutorials**, and **phishing simulation exercises** can also educate non-technical users and reduce real-world risks. With these enhancements, the project can evolve into a powerful, enterprise-grade security solution capable of protecting millions of users from phishing attacks.

REFERENCES

1. Abu-Nimeh, T., Nappa, D., Wang, X., & Nair, S. — *A comparison of machine learning techniques for phishing detection.* (eCrime / ACM)

Retrived from : https://dl.acm.org/doi/10.1145/1299015.1299021

2. Xiang, G., Hong, J., Rosemblat, G., & Cranor, L. — *CANTINA+: A feature-rich ML framework for detecting phishing web sites.* (TSEC/ACM).

Retrived from : https://dl.acm.org/doi/10.1145/2019599.2019606

3. Ma, J., Saul, L. K., Savage, S., & Voelker, G. M. — *Beyond Blacklists: Learning to detect malicious web sites from suspicious URLs.* (KDD 2009)

Retrived from : https://cseweb.ucsd.edu/~jtma/papers/beyondbl-kdd2009.pdf

4. Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. — *Machine learning based phishing detection from URLs.* Expert Systems With Applications (2019)

Retrived from: https://www.sciencedirect.com/science/article/abs/pii/S0957417418306067

5. Thomas, K., Grier, C., Ma, J., Paxson, V., & Song, D. — *Design and evaluation of a real-time URL spam filtering service.* (Related works cited in NDSS/CCS — useful background).

Retrived from: https://www.usenix.org/system/files/conference/cset18/cset18-paper-marchal.pdf

6. Chen, T. & Guestrin, C. — *XGBoost: A scalable tree boosting system.* (arXiv / ACM, 2016) — (core ML algorithm reference used widely).

Retrived from: https://arxiv.org/pdf/1603.02754

7. PhishTank — *Community phishing feed / API (Cisco Talos).*

Retrived from : https://www.phishtank.net/

8. LeCun, Bengio & Hinton — *Deep Learning (Nature review, 2015)* — canonical DL reference from : https://www.cs.toronto.edu/~hinton/absps/NatureDeepReview.pdf

GLOSSARY

1. **Phishing** – A cyber-attack technique where attackers create fake websites, URLs, or messages to trick users into revealing personal or financial information.
2. **URL (Uniform Resource Locator)** – The web address used to access a specific resource or webpage on the internet.
3. **Machine Learning (ML)** – A branch of Artificial Intelligence that enables systems to learn patterns from data and make predictions.
4. **Deep Learning (DL)** – An advanced subset of ML using multi-layer neural networks to learn complex patterns.
5. **Dataset** – A collection of structured data used for training and testing machine learning models.
6. **Feature Extraction** – Transforming raw URL data into numerical attributes usable by ML algorithms.
7. **Lexical Features** – Text-based characteristics of a URL, including length, digits, symbols, domain structure, prefixes, and suffixes.
8. **Label** – The assigned category of a data point.
    - In this project: 1 = Phishing URL, 0 = Legitimate URL
9. **Preprocessing** – Cleaning, formatting, and transforming raw data to make it suitable for model training.
10. **Training** – Teaching a machine learning model using a dataset to recognize patterns.
11. **Testing** – Evaluating a trained model on unseen data to check performance and accuracy.
12. **Accuracy** – The percentage of correct predictions made by the model.
13. **Confusion Matrix** – A table showing correct and incorrect predictions for each class.
14. **Precision** – The proportion of URLs predicted as phishing that were actually phishing.
15. **Recall** – The proportion of actual phishing URLs correctly identified by the model.
16. **F1-Score** – The harmonic mean of precision and recall, showing overall performance.
17. **Classifier** – A model that categorizes input data into predefined classes (phishing or legitimate).
18. **Logistic Regression** – A classification algorithm predicting probabilities and assigning class labels.

19. **Random Forest** – An ensemble ML algorithm using multiple decision trees to improve accuracy.
20. **Decision Tree** – A tree-like model that splits data based on rules to classify it.
21. **Gradient Boosting** – A technique that builds models sequentially, each improving the previous.
22. **XGBoost** – A high-performance gradient-boosted tree algorithm.
23. **Neural Network** – A computational model inspired by the human brain, used to learn patterns.
24. **Model Evaluation** – Assessing a model's effectiveness using performance metrics.
25. **Data Splitting** – Dividing the dataset into training and testing sets (e.g., 70/30 or 80/20).
26. **False Positive** – A legitimate URL incorrectly classified as phishing.
27. **False Negative** – A phishing URL incorrectly classified as legitimate (most dangerous).
28. **Hyperparameters** – Model settings (e.g., learning rate, depth, number of trees) defined before training.
29. **Overfitting** – When a model learns training data too well, reducing performance on new data.
30. **Underfitting** – When a model fails to learn patterns from the training data, resulting in low accuracy.

APPENDIX A

1. Dataset Details

1.1 Dataset Sample

URL	LABEL
http://secure-login.com	1
https://google.com/	0
http://bankofamerica-login.net	1
https://amazon.in	0
http://update-paypal.com	1

Explanation:

- Label = 1 Phishing URL

- Label = 0 Legitimate URL

2. Feature Extraction

Feature Name	Description
URL Length	Total number of characters in the URL
Number of Digits	Count of numeric characters
Number of Symbols	Count of special characters (e.g., @, -, _)
Domain Type	TLD information (e.g., .com, .org)
Prefix/Suffix	Suspicious words in URL like login, update, secure
Presence of IP	Whether URL uses IP address instead of domain name
HTTPS Check	Whether URL uses https

3. Key Code Snippet for Feature Extraction

```
import pandas as pd
import re

def extract_features(url):
    features = {}
    features['url_length'] = len(url)
    features['num_digits'] = sum(c.isdigit() for c in url)
    features['num_symbols'] = sum(not c.isalnum() for c in url)
    features['https'] = int(url.startswith("https"))
    features['ip_in_url'] = int(bool(re.search(r'\d+\.\d+\.\d+\.\d+', url)))
    return features

url = "http://secure-login.com"
print(extract_features(url))
```

APPENDIX B

1. Machine Learning Models

Model	Description
Logistic Regression	Predicts probability of URL being phishing
Decision Tree	Splits data based on rules to classify URLs
Random Forest	Ensemble of decision trees to improve accuracy
Gradient Boosting	Sequential trees to reduce errors
XGBoost	High-performance gradient boosted tree algorithm

2. Deep Learning Model

- **Neural Network:**
  - **Input layer:** Number of features extracted from URL
  - **Hidden layers:** 2–3 layers with Re LU activation
  - **Output layer:** 1 neuron with Sigmoid activation (Phishing/Legitimate)
  - **Loss function:** Binary Cross-Entropy
  - **Optimizer:** Adam

3. Evaluation Metrics

Metric	Definition
Accuracy	% of correct predictions
Precision	Phishing URLs correctly predicted / All predicted phishing
Recall	Phishing URLs correctly predicted / All actual phishing URLs
F1-Score	Harmonic mean of Precision & Recall
Confusion Matrix	Table of TP, FP, TN, FN counts

4. Sample Confusion Matrix

Actual / Predicted	Phishing (1)	Legitimate (0)
Phishing (1)	95	5
Legitimate (0)	3	97